Programarea calculatoarelor

FMI

Secția Calculatoare și tehnologia informației, anul I

Cursul 7 / 17.11.2025

Programa cursului

1. Introducere
- Algoritmi
- Limbaje de programare.
- Introducere în limbajul C. Structura unui program C

2. Fundamentele limbajului C
- Etapele realizării unui program C
- Tipuri de date fundamentale. Variabile. Constante. Operatori. Expresii. Conversii
- Tipuri derivate de date: tablouri, șiruri de caractere, structuri, uniuni, câmpuri de biți, enumerări, pointeri
- Instrucțiuni de control
- Directive de preprocesare. Macrodefiniții
- Funcții de citire/scriere

3. Fișiere text
- Funcții specifice de manipulare

4. Funcții (1)
- Declarare și definire. Apel.
- Metode de trasmitere a paramerilor.
- Pointeri la funcții

5. Tablouri și pointeri
- **Legătura dintre tablouri și pointeri**
- **Aritmetica pointerilor**
- **Alocarea dinamică a memoriei**
- Clase de memorare

6. Șiruri de caractere
- Funcții specifice de manipulare

7. Fișiere binare
- Funcții specifice de manipulare

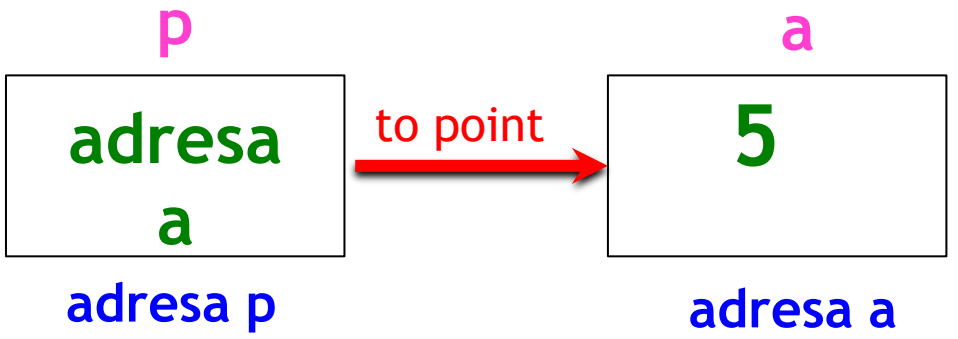
8. Structuri de date complexe și autoreferite
- Definire și utilizare

9. Funcții (2)
- Funcții cu număr variabil de argumente
- Preluarea argumentelor funcției main din linia de comandă
- Programare generică

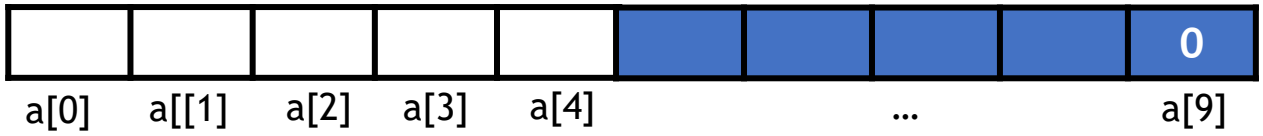
Legătura dintre pointeri și tablouri 1D

- **un pointer =** o variabilă care poate stoca adrese de memorie
  - *exemplu*:

```
int a=5;
int *p;
p = &a;
```



- **un tablou 1D =** un set de valori de același tip memorat la adrese succesive de memorie
  - *exemplu:* int a[10];



Legătura dintre pointeri și tablouri 1D

- adresarea unui element dintr-un tablou cu ajutorul pointerilor
- inițializarea pointerului **p** cu adresa primului element al unui tablou :

```
int *p = v;
p = &v[0];
```

- **numele unui tablou este un pointer (constant) spre primul său element**

- Cum pot să găsesc adresa/ valoarea celui de-al **i**-lea element din vectorul **v** pe baza pointerului **p** (p pointează către adresa de început a tabloului)?

Modelatorul const

- precizează pentru o variabilă inițializată că nu este posibilă modificarea variabilei respectivă. Dacă se încearcă acest lucru se returnează eroare la compilarea programului.

```c
#include <stdio.h>

int main()
{
    const int a = 10;

    a = 5;

    return 0;
}
```

Compilation failed due to following error(s).

```
main.c: In function ‘main’:
main.c:7:7: error: assignment of read-only variable ‘a’
    7 |     a = 5;
      |       ^
```

Modelatorul const

- putem modifica valoarea unei variabile însoțite de modelatorul **const** prin intermediul unui pointer (în mod indirect):

```c
#include <stdio.h>

int main()
{
    const int a = 10;
    int *p = &a;
    *p = 5;

    printf("a = %d \n", a);

    return 0;
}
```

```
main.c: In function ‘main’:
main.c:7:14: warning: initialization discards ‘const’ qualifier
from pointer target type [-Wdiscarded-qualifiers]
    7 |     int *p = &a;
      |              ^
a = 5
```

Pointeri la valori constante

- modelatorul **const** poate preciza pentru un pointer că valoarea variabilei aflate la adresa conținută de pointer nu se poate modifica

```c
int main()
{
    int a = 10;
    const int *p = &a;
    *p = 5;

    printf("a = %d \n", a);

    return 0;
}
```

```
main.c:7:8: error: assignment of read-only location '*p'
    7 |     *p = 5;
      |        ^
```

- putem modifica valoarea pointerului:

```c
int main()
{
    int a = 10, b = 7;
    const int *p = &a;
    p = &b;

    printf("*p = %d \n", *p);

    return 0;
}
```

```
*p = 7
```

Pointeri constanți

- modelatorul **const** poate preciza pentru un pointer că nu poate referi o altă adresă decât cea pe care o conține la inițializare.

```
 3  int main()
 4  {
 5      int a = 10, b;
 6      int* const p = &a;
 7      p = &b;
 8      printf("*p = %d \n", *p);
 9
10      return 0;
11  }
```

```
main.c:7:7: error: assignment of read-only variable ‘p’
    7 |     p = &b;
      |       ^
```

- putem modifica valoarea variabilei aflate la adresa conținută de pointer:

```
 3  int main()
 4  {
 5      int a = 10;
 6      int* const p = &a;
 7      printf("*p = %d \n", *p);
 8      *p = 5;
 9      printf("*p = %d \n", *p);
10
11      return 0;
12  }
```

```
*p = 10
*p = 5
```

Pointeri constanți la valori constante

- modelatorul **const** poate preciza pentru un pointer că:
  - ✓ **nu poate referi o altă adresă decât cea pe care o conține la inițializare**
  - ✓ **nu poate schimba valoare variabilei aflate la adresa pe care o conține**.

```c
#include <stdio.h>

int main()
{
    int a = 10;
    const int* const p = &a;
    printf("*p = %d \n", *p);
    *p = 5;
    int b = 7;
    p = &b;
    printf("*p = %d \n", *p);

    return 0;
}
```

```
main.c:8:8: error: assignment of read-only location '*(const int *)p'
    8 |     *p = 5;
      |        ^
main.c:10:7: error: assignment of read-only variable 'p'
   10 |     p = &b;
      |       ^
```

Pointeri constanți vs valori constante

- diferențele constau în poziționarea modelatorului **const** *înainte* sau *după* caracterul *:
  - ✓ **pointer constant**: **int* const p**;
  - ✓ **pointer la o constantă**: **const int* p;**
  - ✓ **pointer constant la o constantă**: **const int* const p;**

- dacă declarăm o funcție astfel:

void f(const int* p)

atunci valorile din zona de memorie referită de **p** nu pot fi modificate.

Legătura dintre pointeri și tablouri 1D

- adresarea unui element dintr-un tablou cu ajutorul pointerilor
- inițializarea pointerului **p** cu adresa primului element al unui tablou :

```
int *p = v;
p = &v[0];
```

- **numele unui tablou este un pointer (constant) spre primul său element**

- Cum pot să găsesc adresa/ valoarea celui de-al **i**-lea element din vectorul **v** pe baza pointerului **p** (p pointează către adresa de început a tabloului)?

Legătura dintre pointeri și tablouri 1D

- adresarea unui element dintr-un tablou cu ajutorul pointerilor

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int v[] = { 0, 2, 4, 10, 20 };
6      int *p = v;
7      printf("Accesam elementele din v \nprin intermediul lui p:\n");
8      for (int i = 0; i < sizeof(v)/sizeof(int); i++)
9          printf("v[%d]  = %d \n", i, *(p+i));
10     printf("\n");
11
12     p = &v[0];
13     for (int i = 0; i < 5; i++)
14         printf("v[%d]  = %d \n", i, *(p+i));
15
16     return 0;
17 }
```

Legătura dintre pointeri și tablouri 1D

- adresarea unui element dintr-un tablou cu ajutorul pointerilor

```c
#include <stdio.h>

int main()
{
    int v[] = { 0, 2, 4, 10, 20 };
    int *p = v;
    printf("Accesam elementele din v \nprin intermediul lui p:\n");
    for (int i = 0; i < sizeof(v)/sizeof(int); i++)
        printf("v[%d]  = %d \n", i, *(p+i));
    printf("\n");

    p = &v[0];
    for (int i = 0; i < 5; i++)
        printf("v[%d]  = %d \n", i, *(p+i));

    return 0;
}
```

```
Accesam elementele din v
prin intermediul lui p:
v[0]  = 0
v[1]  = 2
v[2]  = 4
v[3]  = 10
v[4]  = 20

v[0]  = 0
v[1]  = 2
v[2]  = 4
v[3]  = 10
v[4]  = 20
```

Legătura dintre pointeri și tablouri 1D

- adresarea unui element dintr-un tablou cu ajutorul pointerilor

  - ✓ **adresa** lui v[i]: **&v[i] = p+i**
  - ✓ **valoarea** lui v[i]: **v[i] = *(p+i)**
  - ✓ comutativitate: v[i] = *(p+i) = *(i+p) = i[v] ?!

Legătura dintre pointeri și tablouri 1D

- adresarea unui element dintr-un tablou cu ajutorul pointerilor

```c
#include <stdio.h>

int main()
{
    int v[] = { 0, 2, 4, 10, 20 };

    printf("Afisam v[i]:\n");
    for (int i = 0; i < sizeof(v)/sizeof(int); i++)
        printf("v[%d]  = %d \n", i, v[i]);
    printf("\n");

    printf("Afisam i[v]:\n");
    for (int i = 0; i < 5; i++)
        printf("v[%d]  = %d \n", i, i[v]);

    return 0;
}
```

```
Afisam v[i]:
v[0]  = 0
v[1]  = 2
v[2]  = 4
v[3]  = 10
v[4]  = 20

Afisam i[v]:
v[0]  = 0
v[1]  = 2
v[2]  = 4
v[3]  = 10
v[4]  = 20
```

Concluzie?

Legătura dintre pointeri și tablouri 1D

- adresarea unui element dintr-un tablou cu ajutorul pointerilor

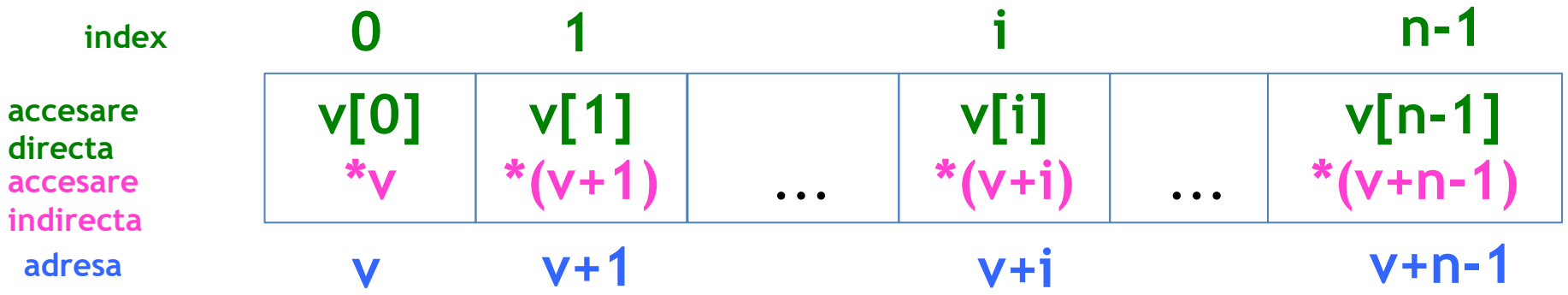
- ***conceptul de tablou nu există în limbajul C***. Numele unui tablou este un pointer (***constant***) spre primul său element.

Legătura dintre pointeri și tablouri 1D

- numele unui tablou este un pointer constant spre primul său element

int v[100]; → v = &v[0];

- elementele unui tablou pot fi accesate prin pointeri:



- operatorul * are prioritate mai mare ca +
- *(v+1) e diferit de *v+1

Legătura dintre pointeri și tablouri 1D

- o expresie cu tablou și indice este echivalentă cu una scrisă ca pointer și distanță de deplasare:

  v[i] = *(v+i)

- **diferența dintre un nume de tablou și un pointer:**
  - ✓ un pointer își poate schimba valoarea:

    p = v și p++ **sunt expresii corecte**

  - ✓ un nume de tablou este un pointer constant (nu își poate schimba valoarea):

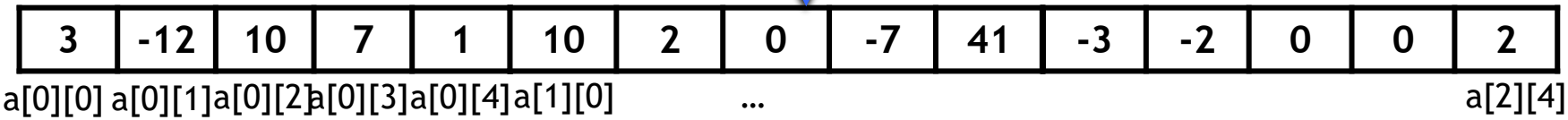
    v = p și v++ **sunt expresii incorecte**

Legătura dintre pointeri și tablouri 2D

```
int a[3][5];

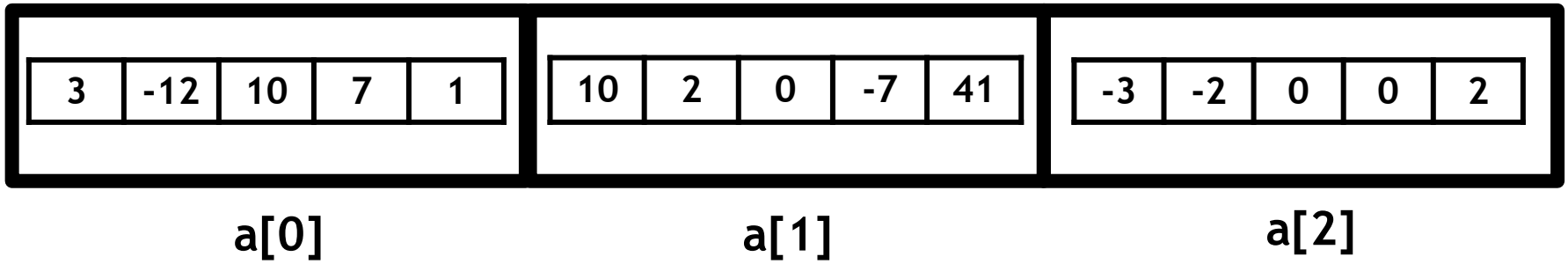
a[1][4] = 41;
```

	0	1	2	3	4
0	3	-12	10	7	1
1	10	2	0	-7	41
2	-3	-2	0	0	2



Reprezentarea în memoria calculatorului a unui tablou bidimensional

tablou bidimensional = tablou de tablouri



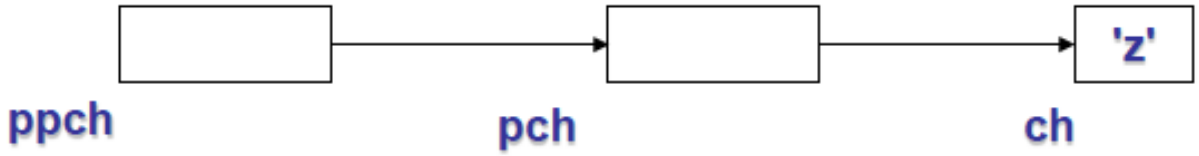
Pointeri la pointeri (pointeri dubli)

Sintaxa

tip **nume_variabilă;

tip = tipul de bază al variabilei de tip pointer dublu nume_variabilă;
***** = operator de indirectare;
nume_variabila = variabila de tip pointer dublu care poate lua ca valori adrese de memorie ale unor variabile de tip pointer.

```
char ch = 'z'; // un caracter
char *pch; // un pointer la caracter
char **ppch; // un pointer la un pointer la caracter
pch = &ch; ppch = &pch;
```




```
printf("%p %p %c",ppch,pch,ch); // 0028FF04 0028FF0B z
```

Pointeri la pointeri

Exemplu:

```c
#include <stdio.h>

int main()
{
    int a = 5, *p = &a, **q = &p;

    printf("Valoarea lui a poate fi obtinuta:\n");
    printf("- direct: a  = %d \n", a);
    printf("- prin p: *p  = %d \n", *p);
    printf("- prin q: **q  = %d \n", **q);
    printf("\n");

    printf("Adresa lui a este %d \n", &a);
    printf("Adresa lui p este %d \n", &p);
    printf("Adresa lui q este %d \n", &q);
    printf("Adresa spre care pointeaza p este %d \n", p);
    printf("Adresa spre care pointeaza q este %d \n", q);
    printf("*q = %d \n", *q);

    return 0;
}
```

Pointeri la pointeri

Exemplu:

```
Valoarea lui a poate fi obtinuta:
- direct: a  = 5
- prin p: *p  = 5
- prin q: **q  = 5

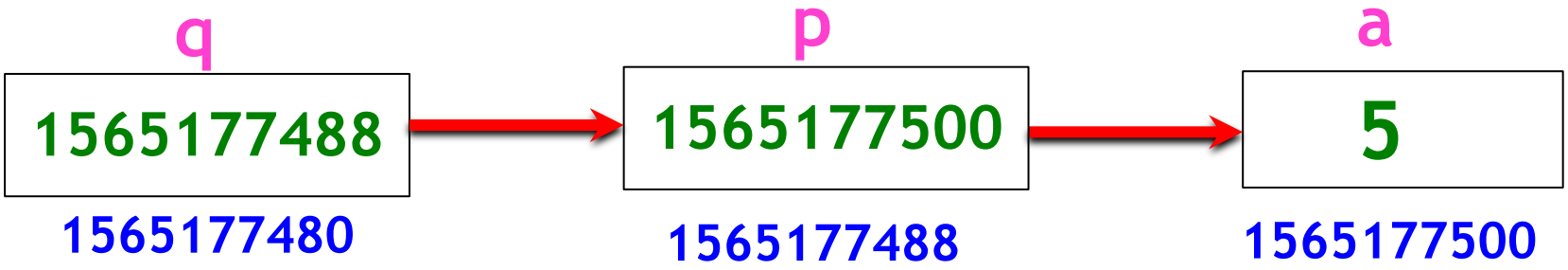
Adresa lui a este 1565177500
Adresa lui p este 1565177488
Adresa lui q este 1565177480
Adresa spre care pointeaza p este 1565177500
Adresa spre care pointeaza q este 1565177488
*q = 1565177500
```

```c
#include <stdio.h>

int main()
{
    int a = 5, *p = &a, **q = &p;

    printf("Valoarea lui a poate fi obtinu
    printf("- direct: a  = %d \n", a);
    printf("- prin p: *p  = %d \n", *p);
    printf("- prin q: **q  = %d \n", **q);
    printf("\n");

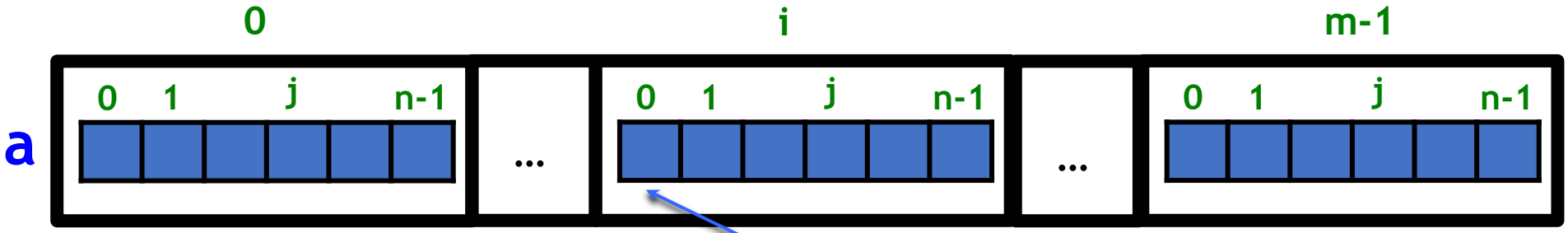
    printf("Adresa lui a este %d \n", &a);
    printf("Adresa lui p este %d \n", &p);
    printf("Adresa lui q este %d \n", &q);
    printf("Adresa spre care pointeaza p este %d \n", p);
    printf("Adresa spre care pointeaza q este %d \n", q);
    printf("*q = %d \n", *q);
```



Legătura dintre pointeri și tablouri 2D

tablou bidimensional = tablou de tablouri

Cazul general: int a[m][n];



Reprezentarea în memoria calculatorului a unui tablou bidimensional

a[i] este un tablou unidimensional. La ce adresă începe a[i]?

Legătura dintre pointeri și tablouri 2D

tablou bidimensional = tablou de tablouri

```c
#include <stdio.h>

int main()
{
    int a[5][5];

    printf("Adresa de inceput a tabloului a: %p\n", a);
    for(int i = 0; i < 5; i++)
        printf("Adresa de inceput a tabloului a[%d]: %p\n", i, &a[i]);

    printf("\n");

    for(int i = 0; i < 5; i++)
        printf("Adresa de inceput a tabloului a[%d]: %d\n", i, &a[i]);

    return 0;
}
```

Legătura dintre pointeri și tablouri 2D

tablou bidimensional = tablou de tablouri

```c
#include <stdio.h>

int main()
{
    int a[5][5];

    printf("Adresa de inceput a tabloului a: %p\n", a);
    for(int i = 0; i < 5; i++)
        printf("Adresa de inceput a tabloului a[%d]: %p\n", i, &a[i]);

    printf("\n");

    for(int i = 0;
        printf("Adr
    return 0;
}
```

```
Adresa de inceput a tabloului a: 0x7ffc047281f0
Adresa de inceput a tabloului a[0]: 0x7ffc047281f0
Adresa de inceput a tabloului a[1]: 0x7ffc04728204
Adresa de inceput a tabloului a[2]: 0x7ffc04728218
Adresa de inceput a tabloului a[3]: 0x7ffc0472822c
Adresa de inceput a tabloului a[4]: 0x7ffc04728240

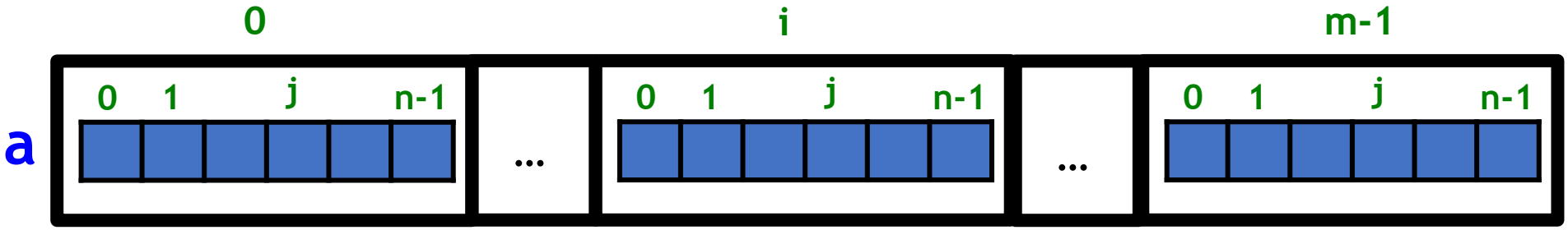
Adresa de inceput a tabloului a[0]: 74613232
Adresa de inceput a tabloului a[1]: 74613252
Adresa de inceput a tabloului a[2]: 74613272
Adresa de inceput a tabloului a[3]: 74613292
Adresa de inceput a tabloului a[4]: 74613312
```

+5*4

Legătura dintre pointeri și tablouri 2D

tablou bidimensional = tablou de tablouri

Cazul general: int a[m][n];

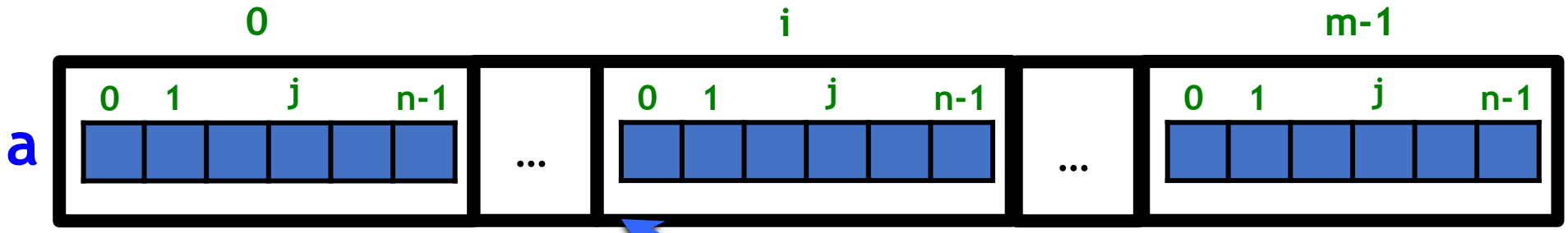


Reprezentarea în memoria calculatorului a unui tablou bidimensional

a[i] este un tablou unidimensional.
1. La ce adresă începe a[i]?

Legătura dintre pointeri și tablouri 2D

- tablou bidimensional = tablou de tablouri
- cazul general: int a[m][n];



Reprezentarea în memoria calculatorului a unui tablou bidimensional

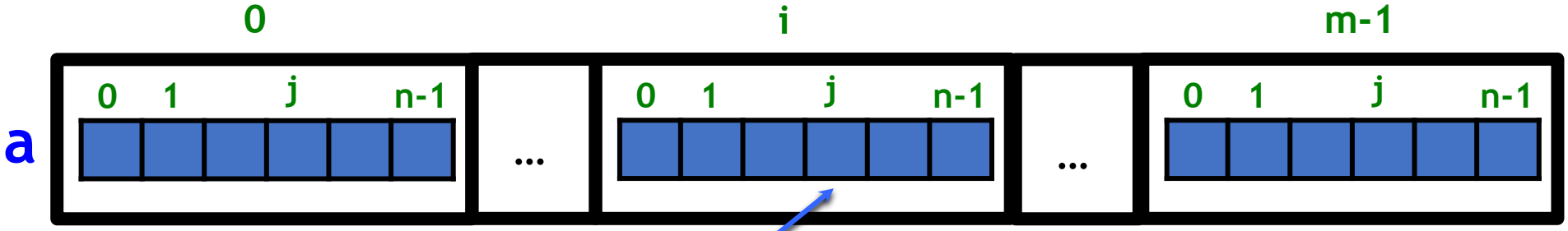
a[i] este un tablou unidimensional.

1. La ce adresă începe a[i]?

R: a[i] începe la adresa $\&a[i] = \&(*(a+i)) = a+i$

Legătura dintre pointeri și tablouri 2D

- tablou bidimensional = tablou de tablouri
- cazul general: int a[m][n];



Reprezentarea în memoria calculatorului a unui tablou bidimensional

a[i] este un tablou unidimensional.

1. a[i] începe la adresa $\&a[i] = \&(*(a+i)) = a+i$

2. Care este adresa lui a[i][j]? Cum o exprim în aritmetica pointerilor în funcție de a, i, j?

Legătura dintre pointeri și tablouri 2D

tablou bidimensional = tablou de tablouri

```c
#include <stdio.h>

int main()
{
    int a[5][5];

    int i = 3;
    for(int j = 0; j < 5; j++)
    {
        printf("Adresa lui a[%d][%d]: %d\n", i, j, &a[i][j]);
        printf("Adresa lui a[%d][%d]: %d\n", i, j, *(a + i) + j);
    }

    return 0;
}
```

Legătura dintre pointeri și tablouri 2D

tablou bidimensional = tablou de tablouri

```c
#include <stdio.h>

int main()
{
    int a[5][5];

    int i = 3;
    for(int j = 0; j < 5; j++)
    {
        printf("Adresa lui a[%d][%d]: %d\n", i, j, &a[i][j]);
        printf("Adresa lui a[%d][%d]: %d\n", i, j, *(a + i) + j);
    }

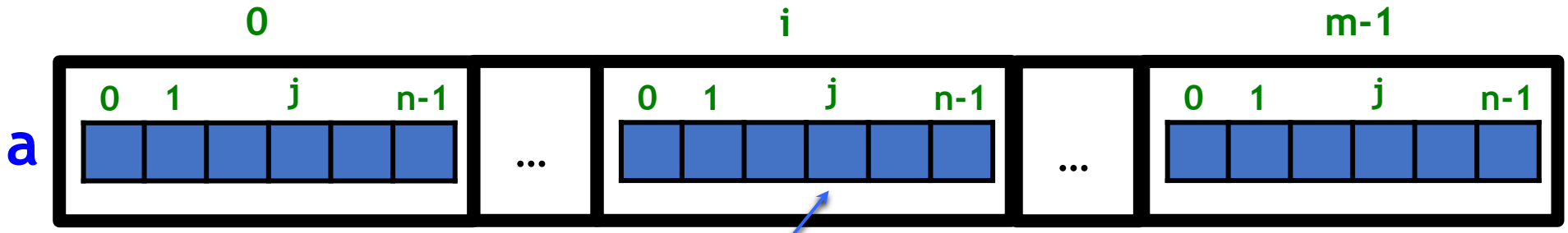
    return 0;
}
```

```
Adresa lui a[3][0]: 1483842828
Adresa lui a[3][0]: 1483842828
Adresa lui a[3][1]: 1483842832
Adresa lui a[3][1]: 1483842832
Adresa lui a[3][2]: 1483842836
Adresa lui a[3][2]: 1483842836
Adresa lui a[3][3]: 1483842840
Adresa lui a[3][3]: 1483842840
Adresa lui a[3][4]: 1483842844
Adresa lui a[3][4]: 1483842844
```



Legătura dintre pointeri și tablouri 2D

- tablou bidimensional = tablou de tablouri
- cazul general: int a[m][n];



Reprezentarea în memoria calculatorului a unui tablou bidimensional

a[i] este un tablou unidimensional.

1. a[i] începe la adresa **&a[i] = &(*(a+i)) = a+i**
2. Adresa lui a[i][j] = &a[i][j] = *(a+i)+j (a este pointer dublu).
3. **Cum exprim valoarea lui a[i][j] în aritmetica pointerilor în funcție de a, i, j?**

Legătura dintre pointeri și tablouri 2D

- **tablou bidimensional = tablou de tablouri**

```c
#include <stdio.h>

int main()
{
    int a[4][4], i, j;

    for(i = 0; i < 4; i++)
        for(j = 0; j < 4; j++)
            a[i][j] = i*j;

    i = 2;
    for(j = 0; j < 4; j++)
    {
        printf("Valoarea lui a[%d][%d]: %d\n", i, j, a[i][j]);
        printf("Valoarea lui a[%d][%d]: %d\n", i, j, *(*(a + i) + j));
    }

    return 0;
}
```

Legătura dintre pointeri și tablouri 2D

- **tablou bidimensional = tablou de tablouri**

```c
#include <stdio.h>

int main()
{
    int a[4][4], i, j;

    for(i = 0; i < 4; i++)
        for(j = 0; j < 4; j++)
            a[i][j] = i*j;

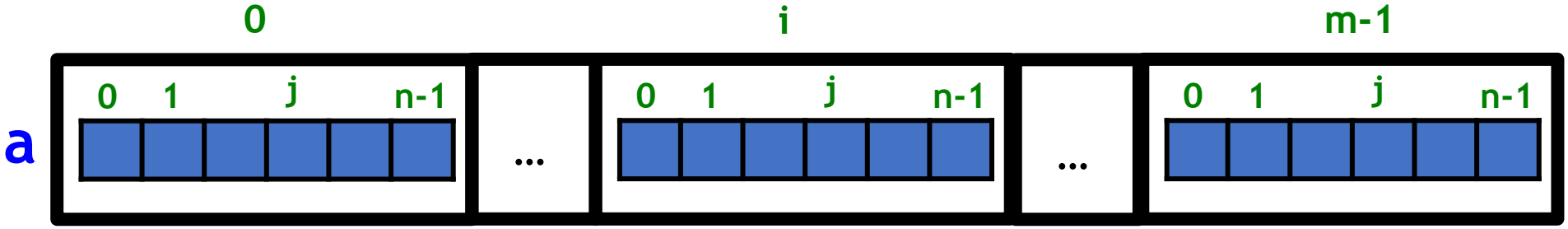
    i = 2;
    for(j = 0; j < 4; j++)
    {
        printf("Valoarea lui a[%d][%d]: %d\n", i, j, a[i][j]);
        printf("Valoarea lui a[%d][%d]: %d\n", i, j, *(*(a + i) + j));
    }

    return 0;
}
```

```
Valoarea lui a[2][0]: 0
Valoarea lui a[2][0]: 0
Valoarea lui a[2][1]: 2
Valoarea lui a[2][1]: 2
Valoarea lui a[2][2]: 4
Valoarea lui a[2][2]: 4
Valoarea lui a[2][3]: 6
Valoarea lui a[2][3]: 6
```

Legătura dintre pointeri și tablouri 2D

- tablou bidimensional = tablou de tablouri
- cazul general: int a[m][n];



Reprezentarea în memoria calculatorului a unui tablou bidimensional

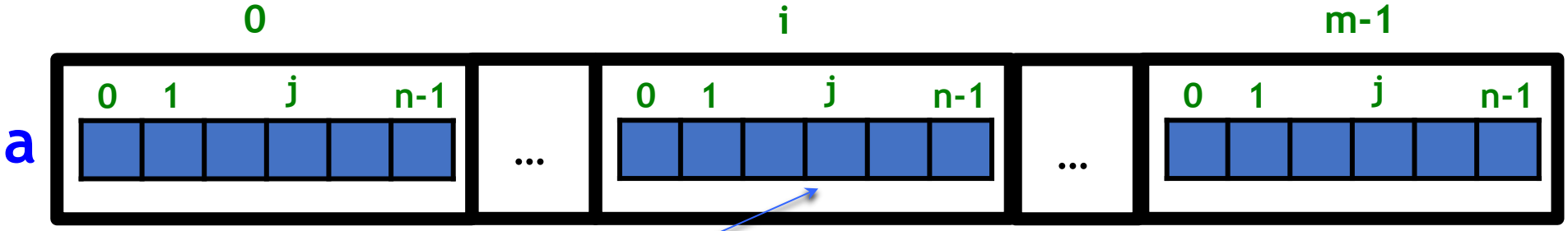
2. Cum exprim adresa lui a[i][j în aritmetica pointerilor în funcție de a, i, j?

R: Adresa lui a[i][j] este **&a[i][j] = *(a+i)+j** (a este pointer dublu).

3. Cum exprim valoarea lui a[i][j] în aritmetica pointerilor în funcție de a, i, j?

Legătura dintre pointeri și tablouri 2D

- tablou bidimensional = tablou de tablouri
- cazul general: int a[m][n];



Reprezentarea în memoria calculatorului a unui tablou bidimensional

Adresa lui a[i][j] este **&a[i][j] = *(a+i)+j** (a este pointer dublu)

Valoarea lui a[i][j] este **a[i][j] = *(*(a+i)+j)**

Legătura dintre pointeri și tablouri 2D

tablou bidimensional = tablou de tablouri

- **caz general:** **int a[m][n];**

✓ Adresa lui a[i][j] este *(a+i)+j (a este pointer dublu)

✓ Valoarea lui a[i][j] este *(*(a+i)+j)

✓ Știu că **a[i] = *(a+i) = i[a].** Atunci a[i][j] se mai poate scrie ca:

1. *(a[i]+j)
2. *(i[a] + j)
3. (*(a+i))[j]
4. i[a][j]
5. j[i[a]]
6. j[a[i]]

Cuprinsul cursului de azi

1. **Legătura dintre pointeri și tablouri**
2. **Aritmetica pointerilor**
3. **Alocarea dinamică a memoriei**

Aritmetica pointerilor

- asupra pointerilor pot fi realizate **operații aritmetice**:
  1) **incrementare** (++)
  2) **decrementare** (--)
  3) **adăugare** a unui întreg (+ sau +=)
  4) **scădere** a unui întreg (- sau -=)
  5) **scădere a unui pointer din alt pointer**
  6) **asignări**
  7) **comparații**

Aritmetica pointerilor

- **inițializarea unui pointer cu adresa primul element al unui tablou**

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int v[5];
    int *pv;

    pv = &v[0];
    printf("Adresa lui v[0] este %x \n", pv);

    pv = v;
    printf("Adresa lui v este %x \n", pv);

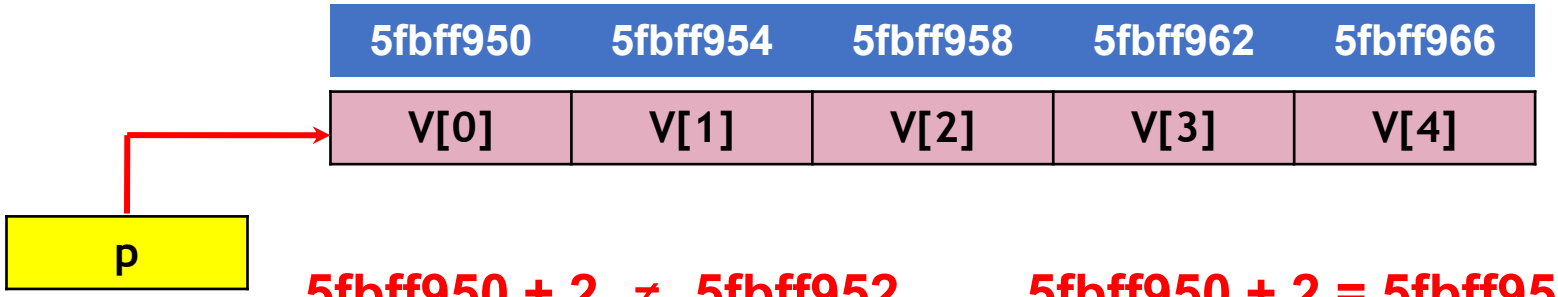
    return 0;
}
```

pv este un pointer care pointeaza către v[0]

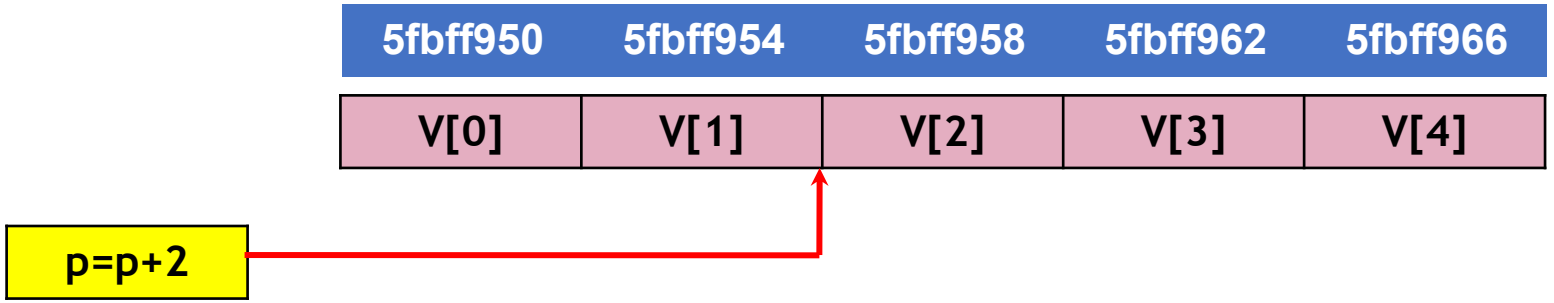
```
Adresa lui v[0] este e285d4e0
Adresa lui v este e285d4e0
```

Aritmetica pointerilor

- **adunarea/scăderea unui număr natural dintr-un pointer**



- adăugarea unui întreg la o adresă de memorie are ca rezultat o nouă adresă de memorie!



Aritmetica pointerilor

Exemplu: adunarea unui număr natural la un pointer

```c
#include <stdio.h>

int main()
{
    int v[5];
    int *p;

    p = &v[0];
    printf("Adresa lui v[0] este %x \n", p);

    p = v;
    printf("Adresa lui v este %x \n", p);

    p = p + 2;
    printf("Adresa spre care pointeaza p acum este %x \n", p);

    return 0;
}
```

```
Adresa lui v[0] este c36c7f00
Adresa lui v este c36c7f00
Adresa spre care pointeaza p acum este c36c7f08
```

Aritmetica pointerilor

Exemplu: adunarea/scăderea unui număr natural dintr-un pointer

```
 5      int v[5];
 6      int *p;
 7
 8      p = &v[0];
 9      printf("Adresa spre care pointeaza acum p este %d \n", p);
10      p += 4;
11      printf("Adresa spre care pointeaza acum p este %d \n", p);
12      p -= 2;
13      printf("Adresa spre care pointeaza acum p este %d \n", p);
14      p++;
15      printf("Adresa spre care pointeaza acum p este %d \n", p);
16      ++p;
17      printf("Adresa spre care pointeaza acum p este %d \n", p);
18      p--;
19      printf("Adresa spre care pointeaza acum p este %d \n", p);
20      --p;
21      printf("Adresa spre care pointeaza acum p este %d \n", p);
```

Aritmetica pointerilor

Exemplu: adunarea/scăderea unui număr natural dintr-un pointer

```
 5      int v[5];
 6      int *p;
 7
 8      p = &v[0];
 9      printf("Adresa spre care pointeaza acum p este %d \n", p);
10      p += 4;
11      printf("Adresa spre care pointeaza acum p este %d \n", p);
12      p -= 2;
13      printf("Adresa spre care pointeaza acum p este %d \n", p);
14      p++;
15      printf("Adresa spre care pointeaza acum p este %d \n", p);
16      ++p;
17      printf("Adresa spre care pointeaza acum p este %d \n", p);
18      p--;
19      printf("Adresa spre care pointeaza acum p este %d \n", p);
20      --p;
21      printf("Adresa spre care pointeaza acum p este %d \n", p);
```

```
Adresa spre care pointeaza acum p este 1247672528
Adresa spre care pointeaza acum p este 1247672544
Adresa spre care pointeaza acum p este 1247672536
Adresa spre care pointeaza acum p este 1247672540
Adresa spre care pointeaza acum p este 1247672544
Adresa spre care pointeaza acum p este 1247672540
Adresa spre care pointeaza acum p este 1247672536
```

Aritmetica pointerilor

Adunarea/scăderea unui număr natural dintr-un pointer:

- adunarea cu n: adresa aflată peste *n* locații de memorie de adresa curentă stocată în pointer (“la dreapta”, se obține adăugând la adresa curentă n*sizeof(*p) octeți) de același tip cu tipul de bază al variabilei de tip pointer
- scăderea cu n: adresa aflată înainte cu *n* locații de memorie de adresa curentă stocată în pointer (“la stânga”, se obține scăzând la adresa curentă n*sizeof(*p) octeți) de același tip cu tipul de bază al variabilei de tip pointer

Aritmetica pointerilor

Exemplu: **scăderea a două variabile de tip pointer**

```c
#include <stdio.h>

int main()
{
    int v[5];
    int *p, *q;

    p = &v[0];
    printf("Adresa spre care pointeaza acum p este %d \n", p);

    q = &v[3];
    printf("Adresa spre care pointeaza acum q este %d \n", q);

    printf("q - p = %d \n", q - p);

    printf("p - q = %d \n", p - q);

    return 0;
}
```

Aritmetica pointerilor

Exemplu: **scăderea a două variabile de tip pointer**

```c
#include <stdio.h>

int main()
{
    int v[5];
    int *p, *q;

    p = &v[0];
    printf("Adresa spre care pointeaza acum p este %d \n", p);

    q = &v[3];
    printf("Adresa spre care pointeaza acum q este %d \n", q);

    printf("q - p = %d \n", q - p);

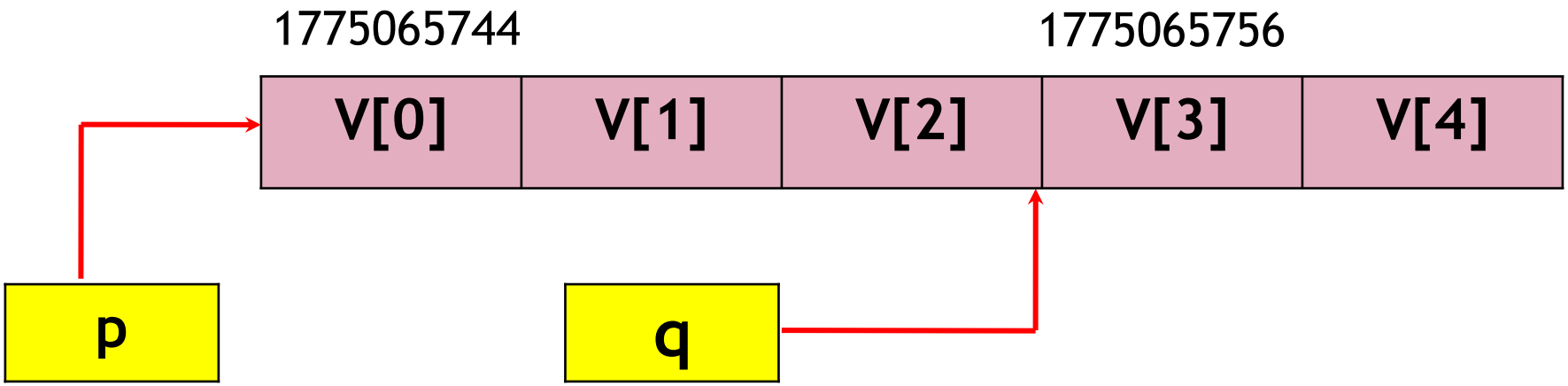
    printf("p - q = %d \n", p - q);

    return 0;
}
```

```
Adresa spre care pointeaza acum p este 1775065744
Adresa spre care pointeaza acum q este 1775065756
q - p = 3
p - q = -3
```

Aritmetica pointerilor

- **scăderea a două variabile de tip pointer**



- în aritmetica pointerilor diferența dintre doi pointeri reprezintă numărul de obiecte de același tip care despart cele două adrese:
  - p – q > 0 înseamnă că p e la dreapta lui q
  - p – q < 0 înseamnă că p e la stânga lui q

Aritmetica pointerilor

Exemplu: **compararea a două variabile de tip pointer**

```c
#include <stdio.h>

int main()
{
    int v[5];
    int *p, *q;

    p = &v[2];
    printf("Adresa spre care pointeaza acum p este %d \n", p);

    q = &v[4];
    printf("Adresa spre care pointeaza acum q este %d \n", q);

    (p > q)? printf("p e la dreapta lui q\n"):printf("p e la stanga lui q\n");

    q = &v[0];
    printf("Adresa spre care pointeaza acum q este %d \n", q);

    p > q ? printf("p e la dreapta lui q\n"):printf("p e la stanga lui q\n");

    return 0;
}
```

Aritmetica pointerilor

Exemplu: **compararea a două variabile de tip pointer**

```c
#include <stdio.h>

int main()
{
    int v[5];
    int *p, *q;

    p = &v[2];
    printf("Adresa spre care pointeaza acum p este %d \n", p);

    q = &v[4];
    printf("Adresa spre care pointeaza acum q este %d \n", q);

    (p > q)? printf("p e la dreapta lui q\n"):printf("p e la stanga lui q\n");

    q = &v[0];
    printf("Adresa spre care pointeaza acum q este %d \n", q);

    p > q ? printf("p e la dreapta lui q\n"):printf("p e la stanga lui q\n");

    return 0;
}
```

```
Adresa spre care pointeaza acum p este 1931020088
Adresa spre care pointeaza acum q este 1931020096
p este la stanga lui q
Adresa spre care pointeaza acum q este 1931020080
p este la dreapta lui q
```

Compararea a două variabile de tip pointer = compararea diferenței lor cu 0

Aritmetica pointerilor

Exemplu: compararea unei variabile de tip pointer cu constanta NULL (0)

```c
#include <stdio.h>

int main()
{
    int v[5];
    int *q = NULL;

    printf("Adresa spre care pointeaza acum q este %d \n", q);

    if (q)
        printf("q contine o adresa valida \n");
    else
        printf("q NU contine o adresa valida \n");

    return 0;
}
```

```
Adresa spre care pointeaza acum q este 0
q NU contine o adresa valida
```

Cuprinsul cursului de azi

1. Legătura dintre pointeri și tablouri
2. Aritmetica pointerilor
3. **Alocarea dinamică a memoriei**

Alocarea dinamică a memoriei

- ***heap***-ul = o zonă predefinită de memorie (de dimensiuni foarte mari) care poate fi accesată de program pentru a stoca date și variabile

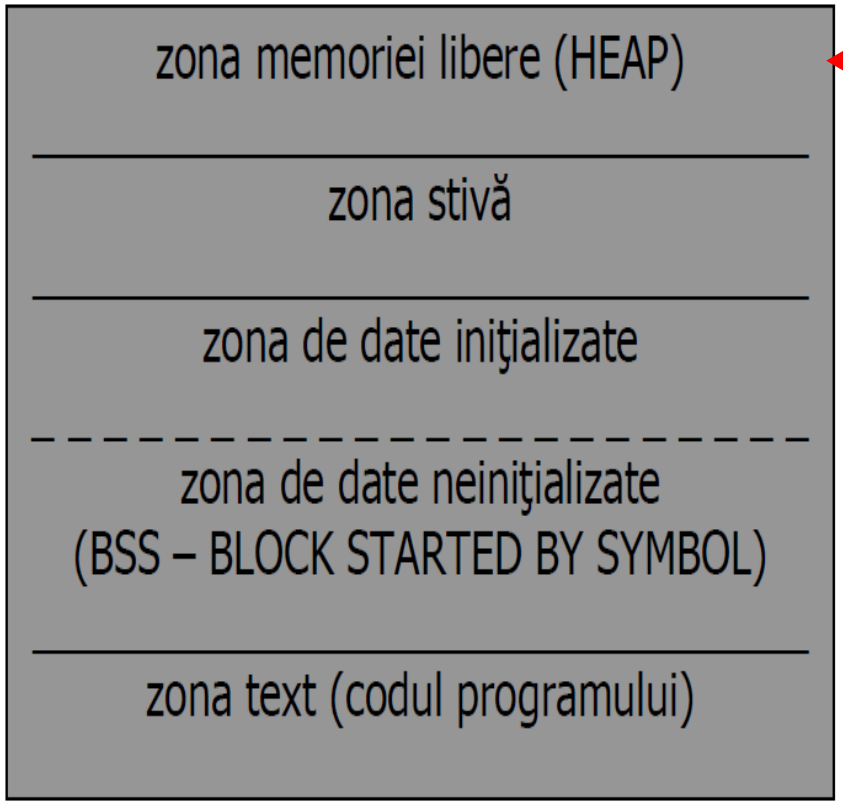
- datele și variabilele pot fi alocate pe *heap* prin apeluri speciale de funcții din biblioteca *stdlib.h*:

  malloc, **calloc**, **realloc**

- zonele de memorie pot să fie dezalocate la cerere prin apelul funcției **free**

- este recomandat ca memoria să fie eliberată în momentul în care datele/variabilele respective nu mai sunt de interes

Harta simplificată a memoriei la rularea unui program



zona de HEAP:
pot aloca dinamic memorie aici și, prin intermediul aritmeticii pointerilor, pot reține informație

Registers

Funcția malloc

- prototipul funcției:

void * malloc(int dimensiune);

unde:

dimensiune = numărul de octeți ceruți a se aloca

- dacă există suficient spaţiu liber în HEAP atunci un bloc de memorie continuu de dimensiunea specificată va fi marcat ca ocupat, iar **funcţia malloc va returna un pointer ce conține adresa de început a acelui bloc.** Dacă nu există suficient spațiu liber funcția malloc întoarce NULL.

- accesarea blocului alocat se realizează printr-un pointer (din STACK/ stivă) către adresa de început a blocului (din HEAP).

Funcția malloc

- prototipul funcției:

void * malloc(int dimensiune);

unde:

dimensiune = numărul de octeți ceruți a se aloca

- tipul generic void * returnat de funcția malloc face obligatorie utilizarea unei conversii de tip atunci când respectivul pointer este asignat unui pointer de tip obișnuit

- pointerul în care păstrăm adresa returnată de malloc va fi plasat în zona de memorie statică

Funcția malloc

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int a = 0;
    int *p = &a;

    printf("Adresa lui a este %d \n", &a);
    printf("Adresa lui p este %d \n", &p);

    printf("Cerere alocare memorie in HEAP \n");
    p = (int*) malloc(5*sizeof(int));
    if (p == NULL)
    {
        printf("Nu exista spatiu liber in HEAP \n");
        exit (0);
    }
    else
        printf("p pointeaza catre adresa %d din HEAP \n", p);
    for(int i = 0; i < 5; i++)
        p[i] = i;
    free(p);

    return 0;
}
```

Funcția malloc

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int a = 0;
    int *p = &a;

    printf("Adresa lui a este %d \n", &a);
    printf("Adresa lui p este %d \n", &p);

    printf("Cerere alocare memorie in HEAP \n");
    p = (int*) malloc(5*sizeof(int));
    if (p == NULL)
    {
        printf("Nu exista spatiu liber in HEAP \n");
        exit (0);
    }
    else
        printf("p pointeaza catre adresa %d din HEAP \n", p);
    for(int i = 0; i < 5; i++)
        p[i] = i;
    free(p);

    return 0;
}
```

```
Adresa lui a este 911972040
Adresa lui p este 911972032
Cerere alocare memorie in HEAP
p pointeaza catre adresa 1897360672 din HEAP
```

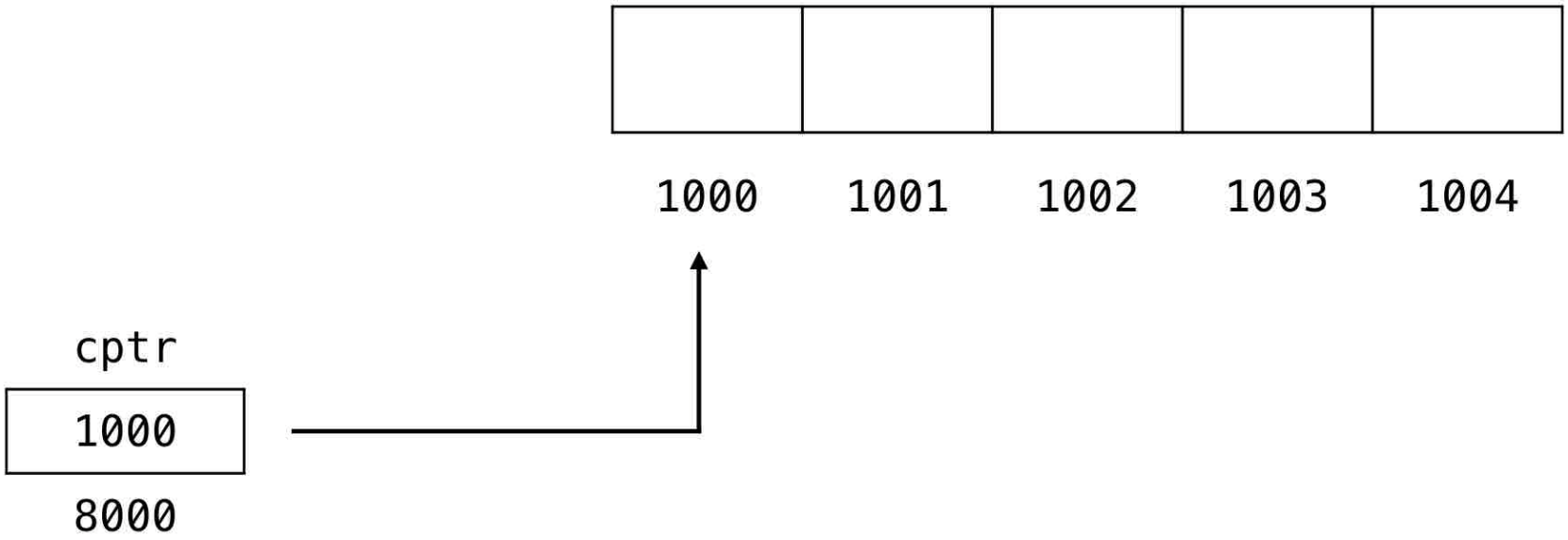
Funcția malloc

Observații:

- ✓blocurile alocate în zona de memorie dinamică **nu au nume**
- ✓**mod de acces**: adresa de memorie
- ✓accesarea blocului de memorie se realizează prin intermediul unui pointer în care păstrăm adresa de început
- ✓orice bloc de memorie alocat dinamic ***trebuie eliberat*** înainte să se încheie execuţia programului. Funcția **free** permite eliberarea memoriei (parametru: adresa de început a blocului).

Funcția malloc

```
char *cptr = (char *) malloc (5 * sizeof(char));
```



dyclassroom.com

https://www.dyclassroom.com/c/c-dynamic-memory-allocation-malloc-function

Funcția malloc

```c
#include <stdio.h>
#include <stdlib.h>

int citire(int *v)
{
    int n, i;
    printf("n = "); scanf("%d", &n);
    v = (int*)malloc(n*sizeof(int));
    for(i = 0; i < n; i++)
        scanf("%d", &v[i]);
    return n;
}

int main()
{
    int n, *p = NULL;
    n = citire(p);
    for(int i = 0; i < n; i++)
        printf("p[%d] = %d ", i, p[i]);
    free(p);

    return 0;
}
```

O funcție pentru citirea unui tablou unidimensional:

1) se citește numărul de elemente,
2) se aloca dinamic tabloul și
3) se citesc elementele tabloului.

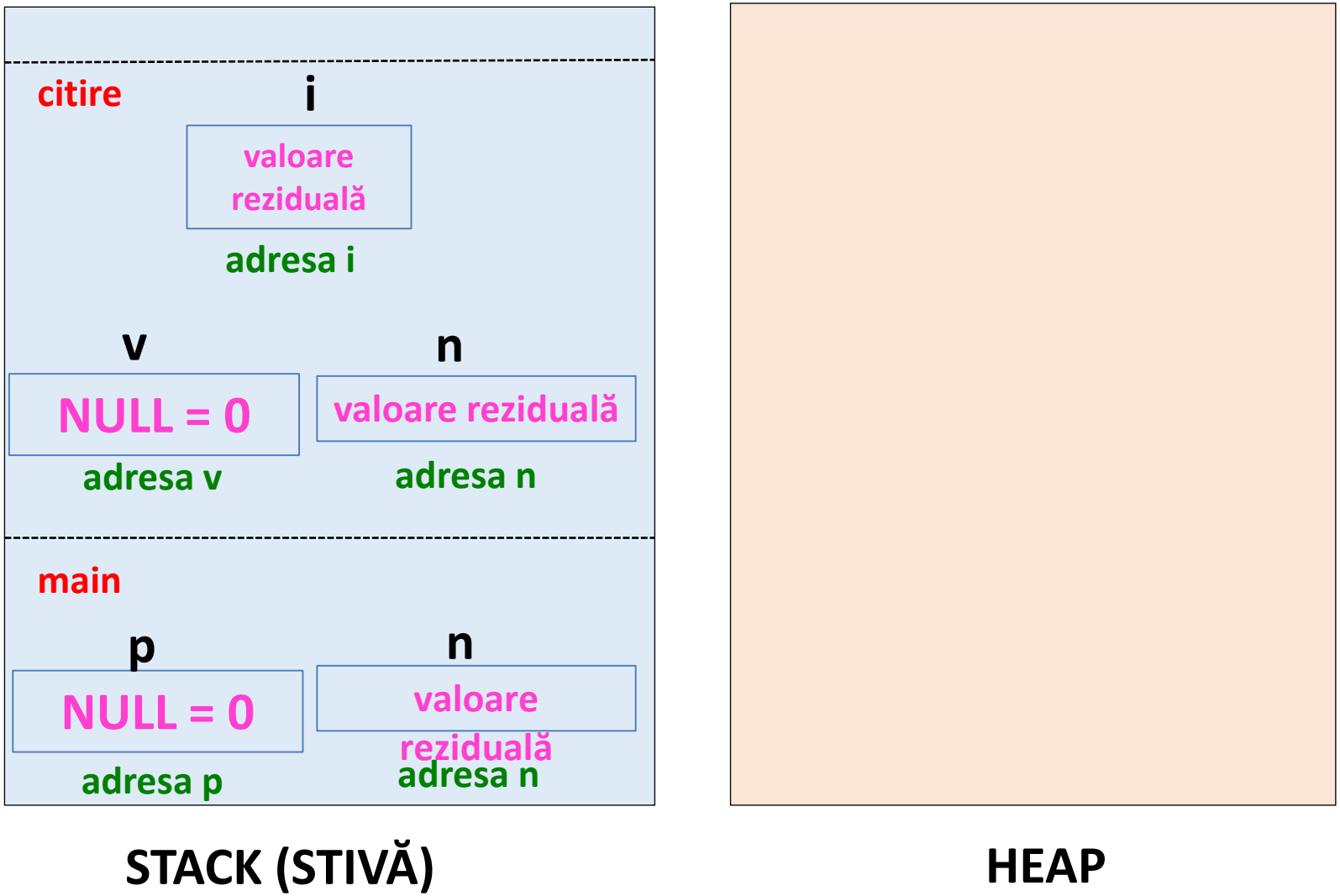
```
n = 3
12 5 3


...Program finished with exit code 139
```

?

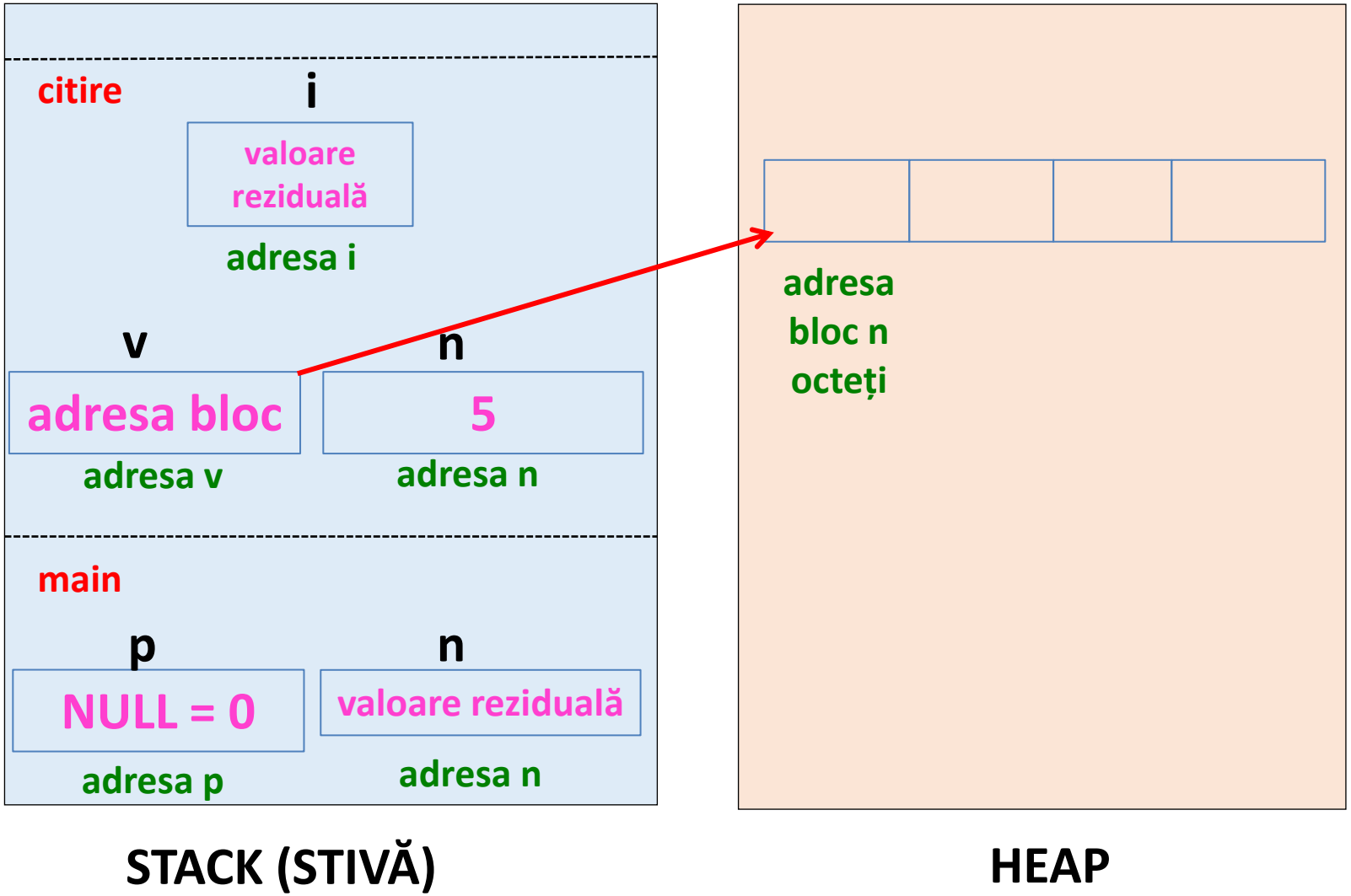
Funcția malloc

Exemplu: Funcție pentru citirea unui tablou unidimensional



Funcția malloc

Exemplu: Funcție pentru citirea unui tablou unidimensional

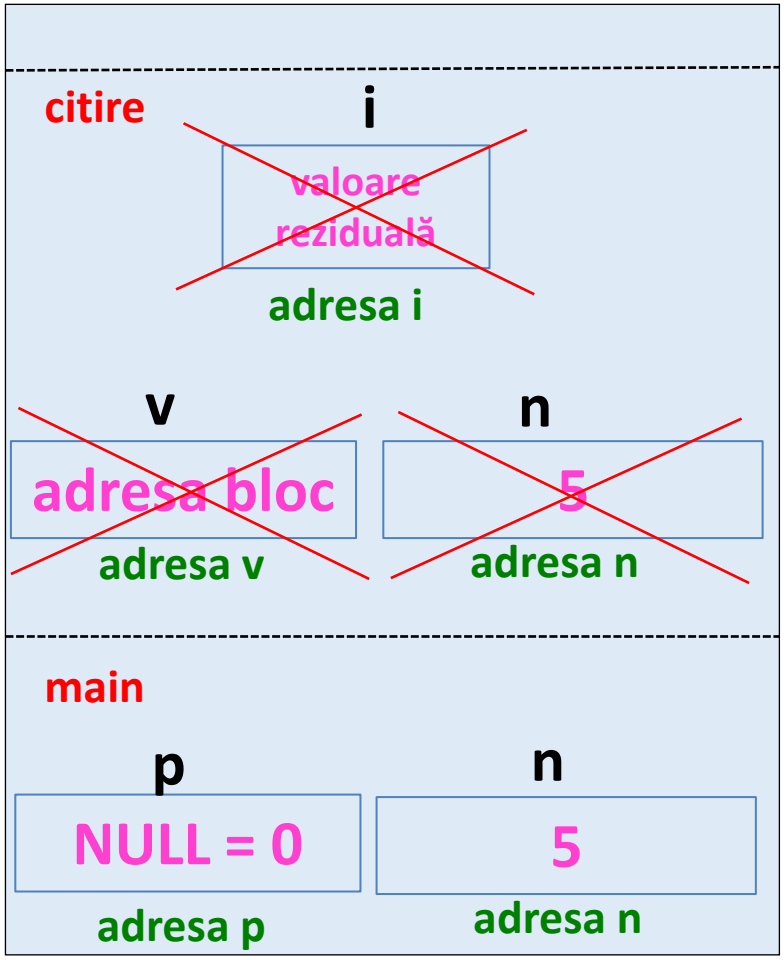


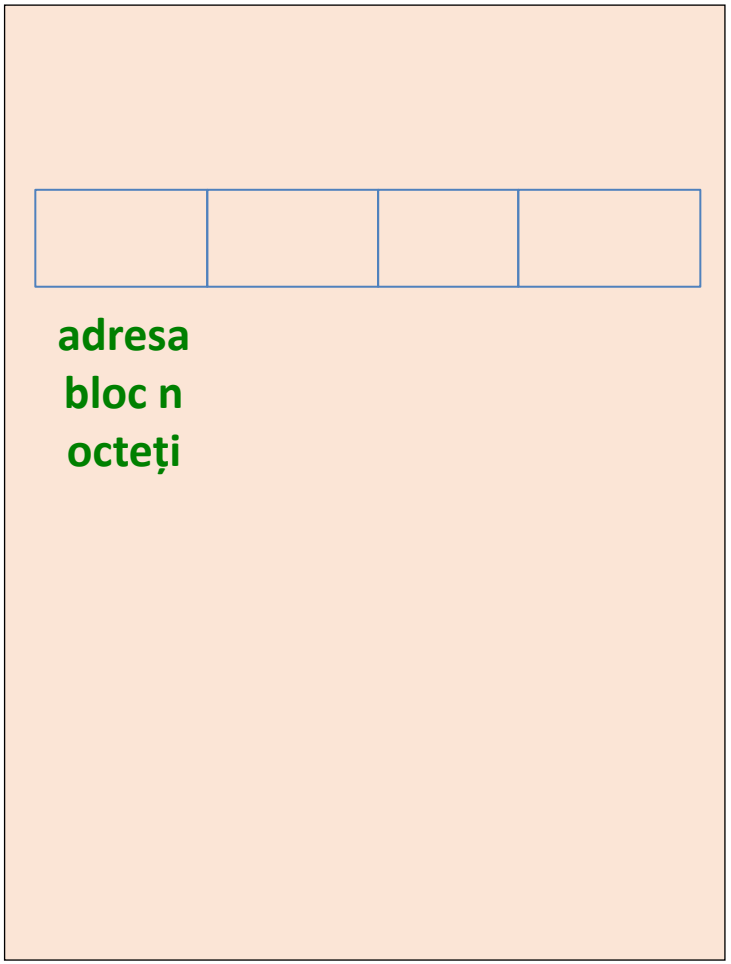
v este copie a lui p

Funcția malloc

Exemplu: Funcție pentru citirea unui tablou unidimensional

v se distruge, se întoarce 5



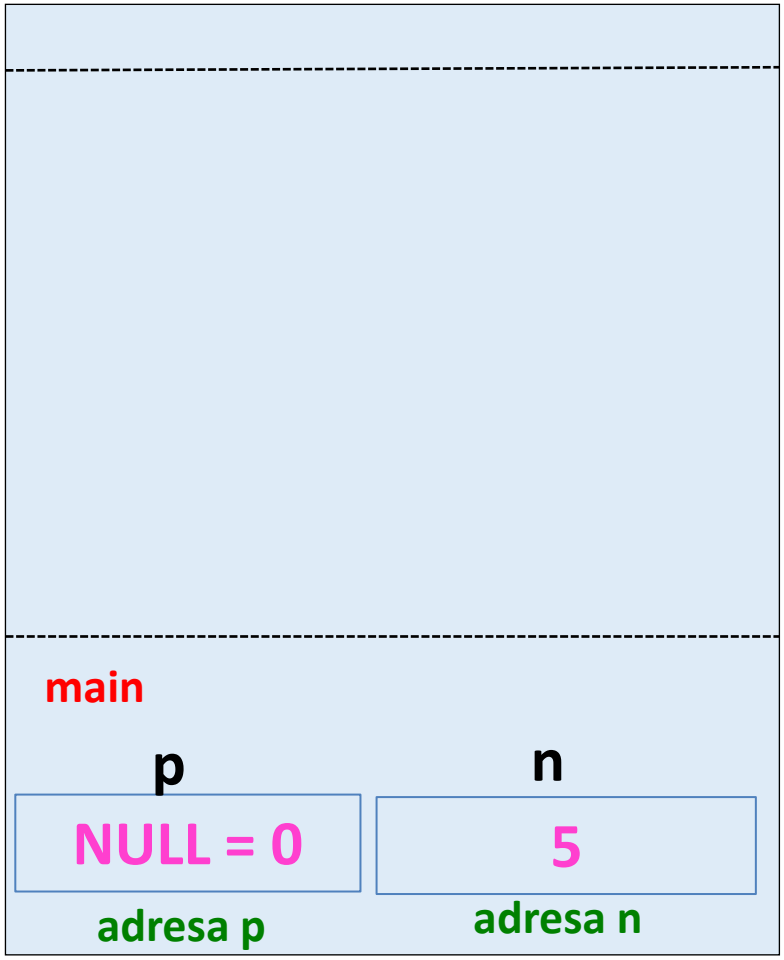


STACK (STIVĂ)

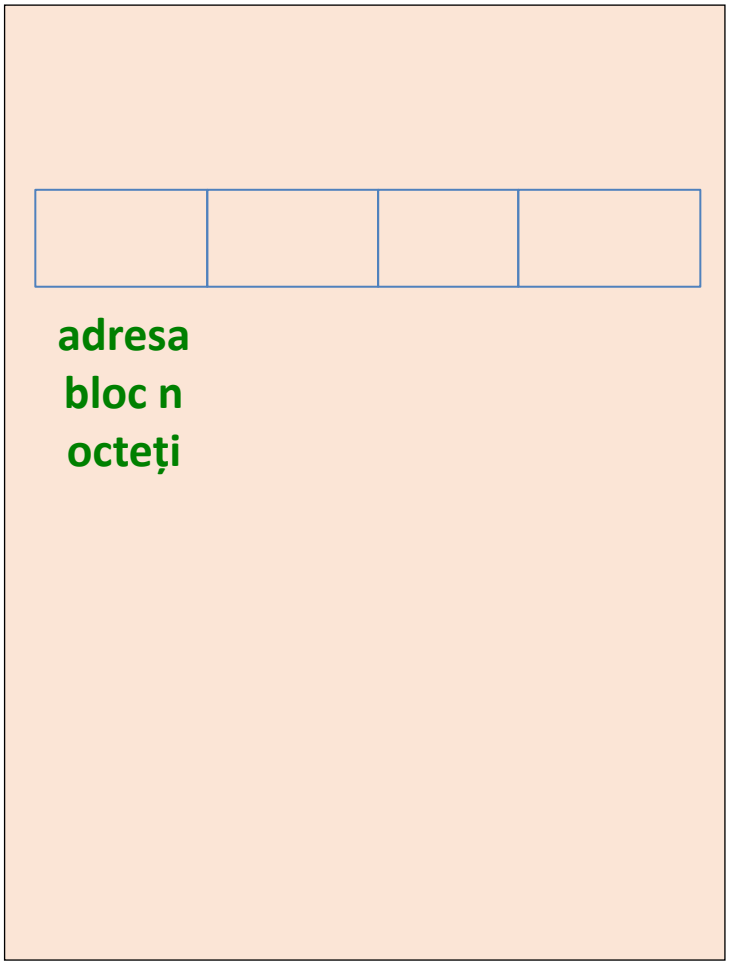
HEAP

Funcția malloc

Exemplu: Funcție pentru citirea unui tablou unidimensional



STACK (STIVĂ)



HEAP

Funcția malloc

```c
#include <stdio.h>
#include <stdlib.h>

int citire1(int **v)
{
    int n, i;
    printf("n = "); scanf("%d", &n);
    *v = (int*)malloc(n*sizeof(int));
    for(i = 0; i < n; i++)
        scanf("%d", &(*v)[i]);
    return n;
}

int main()
{
    int n, *p = NULL;
    n = citire1(&p);
    for(int i = 0; i < n; i++)
        printf("p[%d] = %d ", i, p[i]);

    free(p);

    return 0;
}
```

O funcție pentru citirea unui tablou unidimensional:

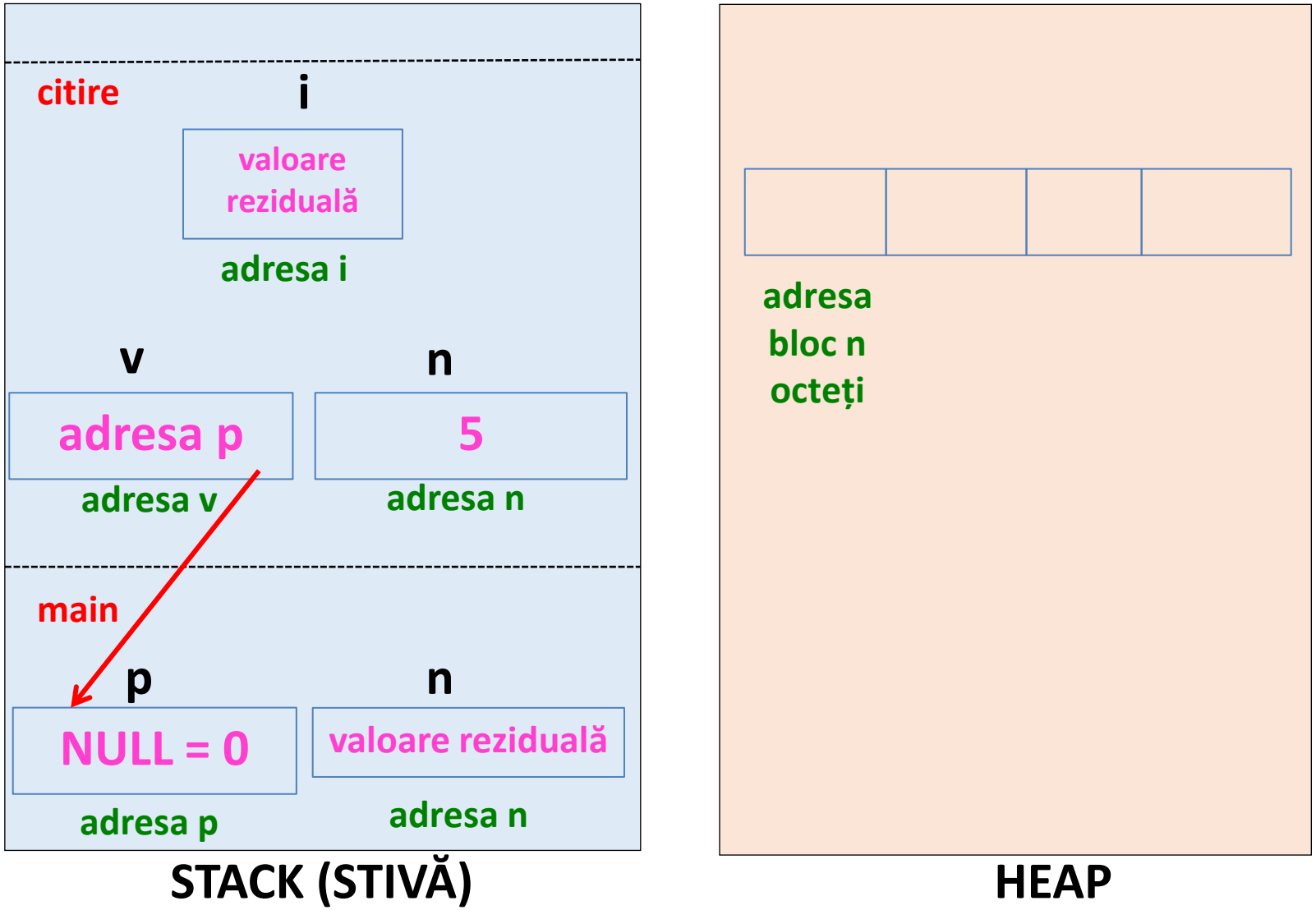
1) se citește numărul de elemente,
2) se aloca dinamic tabloul și
3) se citesc elementele tabloului.

```
n = 4
3 5 7 4
p[0] = 3 p[1] = 5 p[2] = 7 p[3] = 4

...Program finished with exit code 0
```

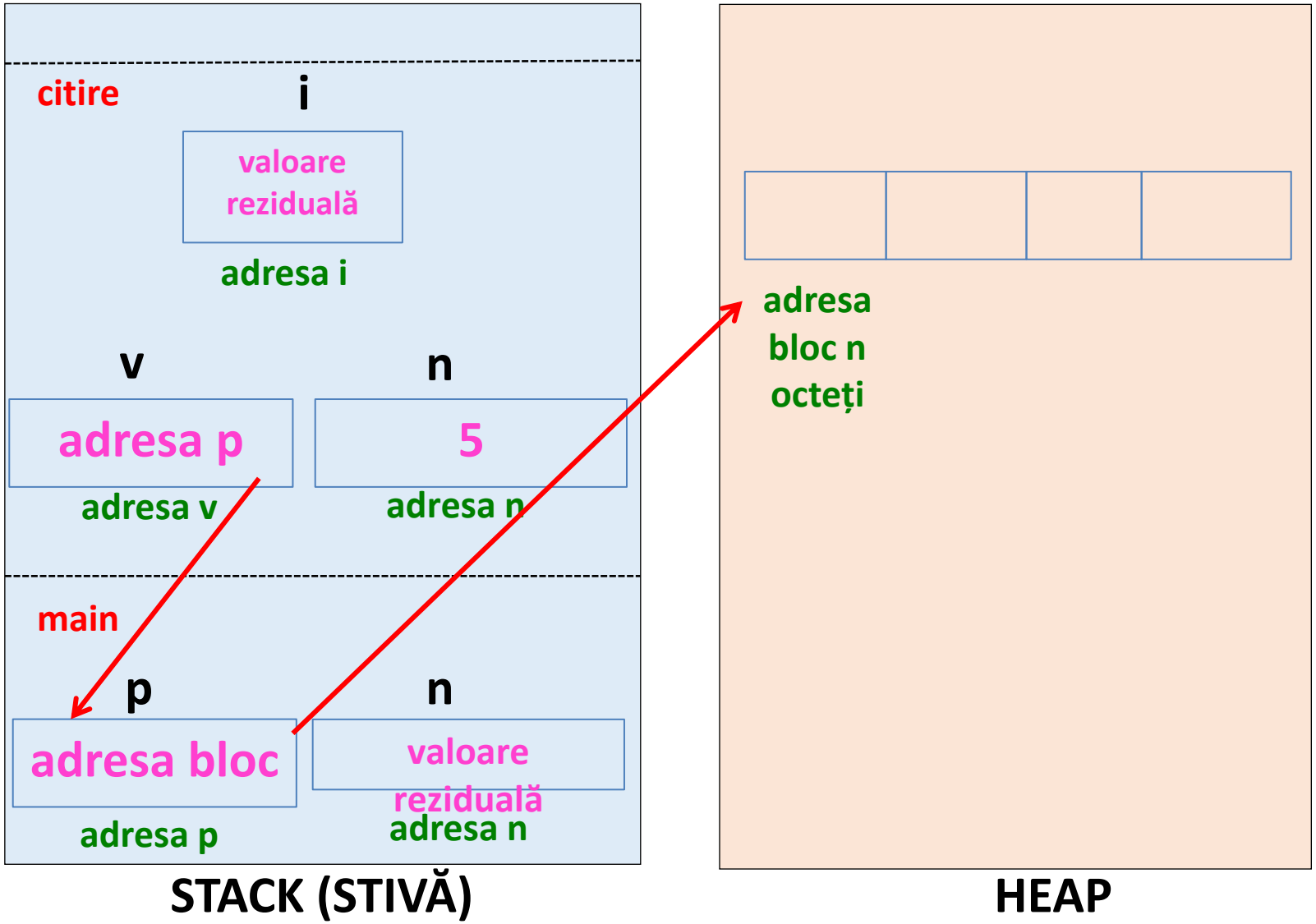
Funcția malloc

- Ex. Funcție pentru citirea unui tablou unidimensional



Funcția malloc

- Ex. Funcție pentru citirea unui tablou unidimensional



Funcția calloc

- prototipul funcției:

void * calloc(int numar, int dimensiune);

unde:

numar = numărul de blocuri/elemente a se aloca

dimensiune = numărul de octeți ceruți pentru fiecare bloc

- dacă există suficient spaţiu liber în HEAP atunci un bloc de memorie continuu de dimensiunea specificată va fi marcat ca ocupat, iar funcţia **calloc va returna un pointer ce conține adresa de început a acelui bloc**
- dacă nu există suficient spațiu liber funcția calloc întoarce NULL
- **diferența față de malloc**: funcția calloc inițializează toate blocurile cu 0

Funcția calloc

```
 6      int n, i, *pi = NULL;
 7      double *pd = NULL;
 8      char *pc = NULL;
 9
10      scanf("%d", &n);
11      pi = (int*) calloc(n, sizeof(int));
12      printf("\nAfisare adrese + valori vector de int: \n");
13      for(i = 0; i < n; i++)
14          printf("%x %d ", pi + i, pi[i]);
15
16      pd = (double*) calloc(n, sizeof(double));
17      printf("\n\nAfisare adrese + valori vector de double: \n");
18      for(i = 0; i < n; i++)
19          printf("%x %.2f ", pd + i, pd[i]);
20
21      pc = (char*) calloc(n, sizeof(char));
22      printf("\n\nAfisare adrese + valori vector de char: \n");
23      for(i = 0; i < n; i++)
24          printf("%x %d ", pc + i, pc[i]);
25
```

Funcția calloc

```
 6      int n, i, *pi = NULL;
 7      double *pd = NULL;
 8      char *pc = NULL;
 9
10      scanf("%d", &n);
11      pi = (int*) calloc(n, sizeof(int));
12      printf("\nAfisare adrese + valori vector de int: \n");
13      for(i = 0; i < n; i++)
14          printf("%x %d ", pi + i, pi[i]);
15
16      pd = (double*) calloc(n, sizeof(double));
17      printf("\n\nAfisare adrese + valori vector de double: \n");
18      for(i = 0; i < n; i++)
19          printf("%x %.2f ", pd + i, pd[i]);
20
21      pc = (char*) calloc(n, sizeof(char));
22      printf("\n\nAfisare adrese + valori vector de char: \n");
23      for(i = 0; i < n; i++)
24          printf("%x %d ", pc + i, pc[i]);
25
```

```
3

Afisare adrese + valori vector de int:
bd38bd20 0 bd38bd24 0 bd38bd28 0

Afisare adrese + valori vector de double:
bd38c150 0.00 bd38c158 0.00 bd38c160 0.00

Afisare adrese + valori vector de char:
bd38c170 0 bd38c171 0 bd38c172 0
```

Funcția realloc

- prototipul funcției:

void * realloc(void *p, int dimensiune);

unde:

p = un pointer (începutul unui bloc de memorie pe care vreau să îl redimensionez - de obicei avem nevoie de mai multă memorie)

dimensiune = numărul de octeți ceruți pentru alocare

- dacă există suficient spaţiu liber în HEAP atunci un bloc de memorie continuu de dimensiunea specificată va fi marcat ca ocupat, iar funcţia **realloc va returna un pointer ce conține adresa de început a acelui bloc. Tot conținutul blocului de memorie inițial se copiază.**
- dacă nu există suficient spațiu liber realloc întoarce NULL.

Funcția realloc

```
 6      int *a, *aux;
 7      a = (int*) malloc(20 * sizeof(int));
 8      if (!a)
 9      {
10         printf("Nu se poate aloca memorie. \n");
11         exit (0);
12      }
13
14      aux = (int*) realloc(a, 100 * sizeof(int));
15      if (!aux)
16      {
17         printf("Nu se poate redimensiona blocul a. \n");
18         free(a);
19         exit (0);
20      }
21      else
22      {
23          printf("\nRedimensionare reusita. \n");
24          a = aux;
25      }
26
27      free(a);
```

```
Redimensionare reusita.


...Program finished with exit code 0
```

Funcția free

- prototipul funcției:

void free(void *p);

unde:

p reprezinta un pointer (începutul unui bloc de memorie pe care vrem să-l eliberăm)

- funcția **free** eliberează zona de memoria alocată dinamic a cărei adresă de început este dată de **p**. Zona de memorie dezalocată este marcată ca fiind disponibilă pentru o nouă alocare.

- un bloc de memorie nu trebuie eliberat de mai multe ori.

Alocarea dinamică a memoriei

- principalul avantaj al folosirii alocării dinamice este gestionarea eficientă a resurselor memoriei:
  - memoria necesară este alocată în timpul execuției programului (când e nevoie) și nu la compilarea programului

Alocare dinamică – aplicații

Exemplu: Se citesc de la tastatură un număr n și apoi n numere întregi. Să se afișeze numerele în ordinea inversă a citirii.

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p, n, i;
    printf("n = "); scanf("%d", &n);
    p = (int*) malloc(n * sizeof(int));
    for(i = 0; i < n; i++)
        scanf("%d", p + i);
    for(i = n - 1; i >= 0; i--)
        printf("%d ", *(p + i));

    free(p);

    return 0;
}
```

```
n = 5
3 5 2 0 7
7 0 2 5 3
```

Alocare dinamică – aplicații

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *v,n,i,s=0,p=1;
    printf("n="); scanf("%d",&n);
    v=(int*)malloc(n*sizeof(int));
    if (v==NULL)
        printf("\nEroare de alocare.");
    for(i=0;i<n;i++)
    {
        printf("nr %d=",i+1); scanf("%d",v+i);
        s+=*(v+i); p*=*(v+i);
    }
    for(i=0;i<n;i++)
        printf("%d ",*(v+i));
    printf("\ns=%d\np=%d\n",s,p);
    free(v);
    return 0;
}
```

Enunț?

Alocare dinamică – aplicații

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *v,n,i,s=0,p=1;
    printf("n="); scanf("%d",&n);
    v=(int*)malloc(n*sizeof(int));
    if (v==NULL)
        printf("\nEroare de alocare.");
    for(i=0;i<n;i++)
    {
        printf("nr %d=",i+1); scanf("%d",v+i);
        s+=*(v+i); p*=*(v+i);
    }
    for(i=0;i<n;i++)
        printf("%d ",*(v+i));
    printf("\ns=%d\np=%d\n",s,p);
    free(v);
    return 0;
}
```

Enunț?

```
n=3
nr 1=1
nr 2=4
nr 3=7
1 4 7
s=12
p=28
```

Alocare dinamică – aplicații

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *v,n,i,m1,m2;
    scanf("%d",&n);
    v=(int*)malloc(n*sizeof(int));
    if (v==NULL)
        printf("\nEroare de alocare.");
    for(i=0;i<n;i++)
    {
        scanf("%d",v+i);
    }
    m1=m2=*v;
    for(i=0;i<n;i++)
    {
        printf("%d ",*(v+i));
        if(m1<*(v+i)) m1=*(v+i);
        if(m2>*(v+i)) m2=*(v+i);
    }
    printf("\n%d %d",m1,m2);
    free(v);
    return 0;
}
```

Enunț?

Alocare dinamică – aplicații

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *v,n,i,m1,m2;
    scanf("%d",&n);
    v=(int*)malloc(n*sizeof(int));
    if (v==NULL)
        printf("\nEroare de alocare.");
    for(i=0;i<n;i++)
    {
        scanf("%d",v+i);
    }
    m1=m2=*v;
    for(i=0;i<n;i++)
    {
        printf("%d ",*(v+i));
        if(m1<*(v+i)) m1=*(v+i);
        if(m2>*(v+i)) m2=*(v+i);
    }
    printf("\n%d %d",m1,m2);
    free(v);
    return 0;
}
```

Enunț?

```
5 12 4 7 3 9
12 4 7 3 9
12 3
```

Alocare dinamică – aplicații

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p = (int*) malloc(20 * sizeof(int)), i;
    if (p == NULL)
        printf("Eroare la alocare. ");

    for(i = 1; i <= 20; i++)
    {
        *(p + i) = i;
        if(*(p + i) >= 1 && *(p + i) < 10)
            printf("f(%d) = %d\n", i, *(p + i)**(p + i) + 2);
        else
            printf("f(%d) = %d\n", i, *(p + i) - 1);
    }

    free(p);

    return 0;
}
```

Enunț?

Alocare dinamică – aplicații

Enunț: se citește de la tastatură un șir de numere întregi până la întâlnirea lui 0. Să se afișeze numerele în ordinea inversă a citirii.

```
4  void afisare(int *p, int dim){
5      printf("dupa %d realocari: ", dim);
6      for(int i = dim -1; i >= 0; i--)
7          printf("%d\t", *(p + i));
8  }
9
10 int main(){
11     int *p, *aux, i = 0, x;
12     printf("Dati numarul: "); scanf("%d", &x);
13     p = (int*) malloc(sizeof(int));
14     while(x != 0)
15     {
16         p[i] = x;
17         afisare(p, i + 1);
18         i++;
19         p = realloc(p, (i + 1)*sizeof(int));
20         printf("\nDati un numar: "); scanf("%d", &x);
21     }
22     free(p);
```

Ce se întâmplă dacă nu se poate realoca memorie?

p devine NULL (se pierde tot conținutul de până atunci).

Alocare dinamică – aplicații

Enunț: se citește de la tastatură un șir de numere întregi până la întâlnirea lui 0. Să se afișeze numerele în ordinea inversă a citirii.

```
4  void afisare(int *p, int dim){
5      printf("dupa %d realocari: ", dim);
6      for(int i = dim -1; i >= 0; i--)
7          printf("%d\t", *(p + i));
8  }
9
10 int main(){
11     int *p, *aux, i = 0, x;
12     printf("Dati numarul: "); scanf("%d", &x);
13     p = (int*) malloc(sizeof(int));
14     while(x != 0)
15     {
16         p[i] = x;
17         afisare(p, i + 1);
18         i++;
19         aux = realloc(p, (i + 1)*sizeof(int));
20         if (aux)
21             p = aux;
22             else{
23                 printf("Eroare la realocare!");
24                 free(p); exit(0);
25             }
26         printf("\nDati un numar: "); scanf("%d", &x);
27     }
28     free(p);
```

Alocare dinamică – aplicații

Enunț: se citește de la tastatură un șir de numere întregi până la întâlnirea lui 0. Să se afișeze numerele în ordinea inversă a citirii.

```
4  void afisare(int *p, int dim){
5      printf("dupa %d realocari: ", dim);
6      for(int i = dim -1; i >= 0; i--)
7          printf("%d\t", *(p + i));
8  }
9
10 int main(){
11     int *p, *aux, i = 0, x;
12     printf("Dati numarul: "); s
13     p = (int*) malloc(sizeof(i
14     while(x != 0)
15     {
16         p[i] = x;
17         afisare(p, i + 1);
18         i++;
19         aux = realloc(p, (i + 1
20         if (aux)
21             p = aux;
22             else{
23                 printf("Eroare la realocare!");
24                 free(p); exit(0);
25             }
26         printf("\nDati un numar: "); scanf("%d", &x);
27     }
28     free(p);
```

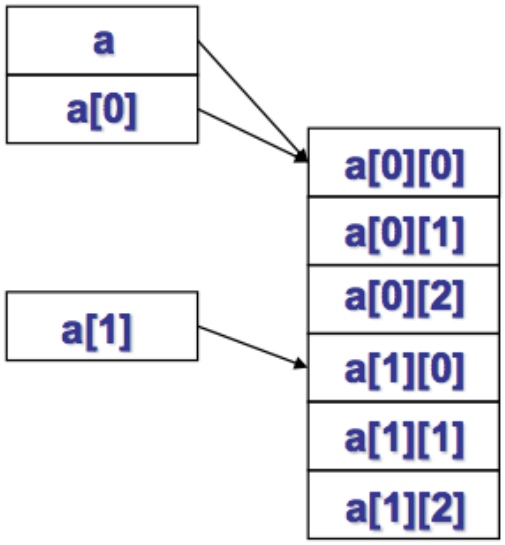
```
Dati numarul: 3
dupa 1 realocari: 3
Dati un numar: 5
dupa 2 realocari: 5     3
Dati un numar: 5
dupa 3 realocari: 5     5       3
Dati un numar: 7
dupa 4 realocari: 7     5       5       3
Dati un numar: 2
dupa 5 realocari: 2     7       5       5       3
Dati un numar: 0
```

Alocare dinamică – aplicații

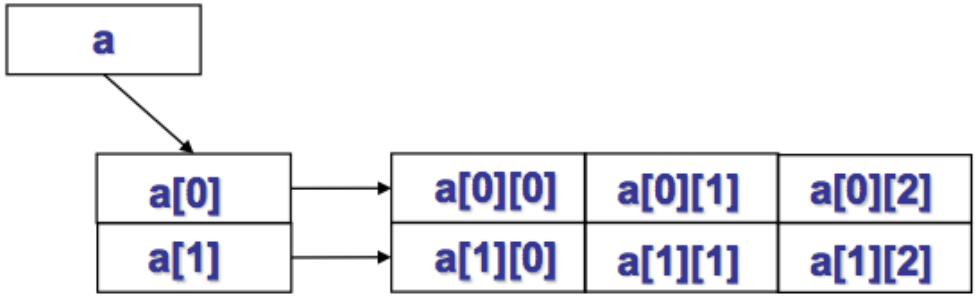
- alocarea dinamică a unui tablou bi-dimensional

Alocarea statică (pe STIVĂ)

```
int a[2][3];
```



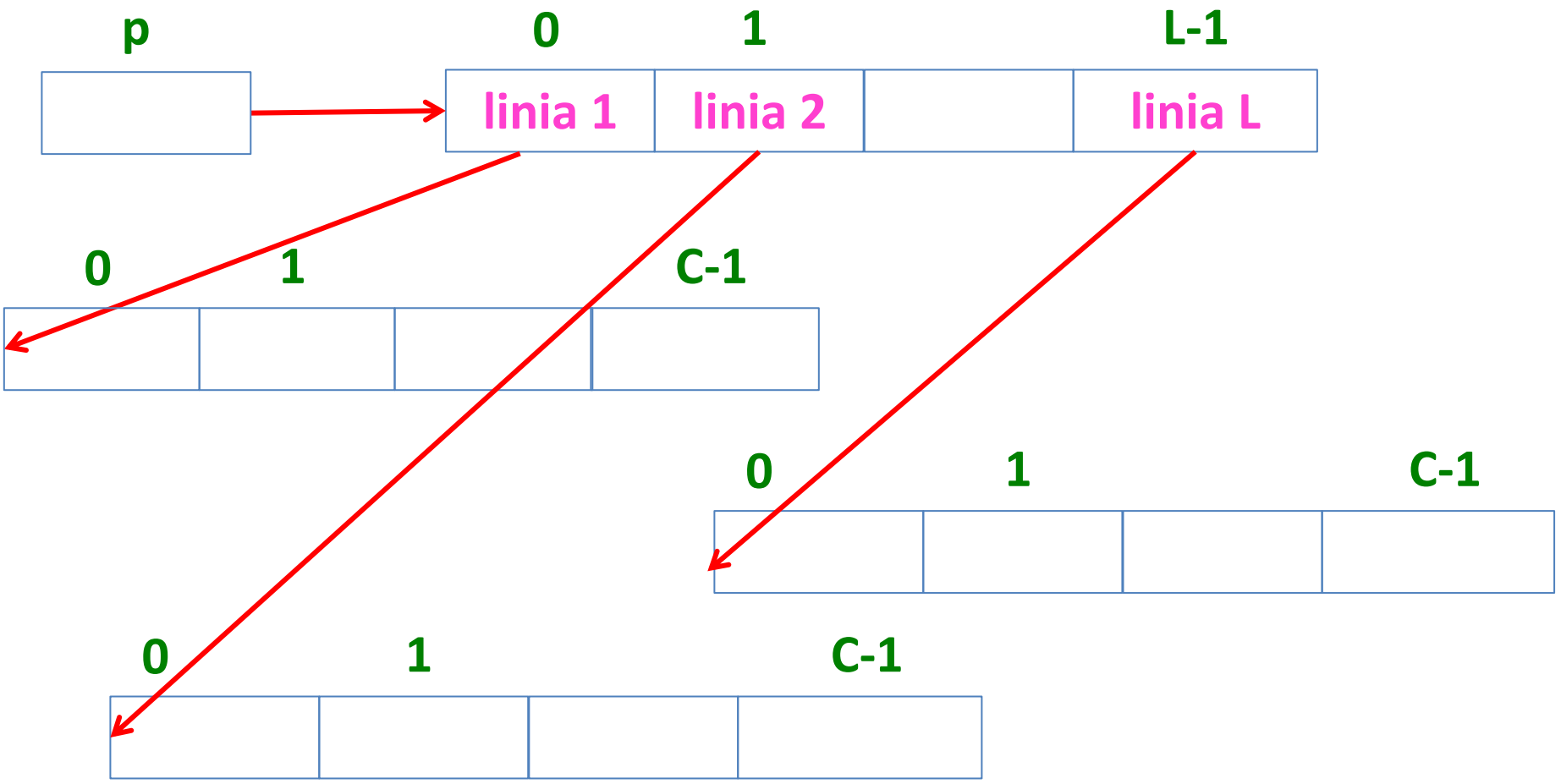
Alocarea dinamică (pe HEAP)



a e pointer dublu
Tipul lui a => int **
Tipul lui a[0] => int *
Tipul lui a[1] => int *

Alocare dinamică – aplicații

- alocarea dinamică a unui tablou bi-dimensional



Alocare dinamică – aplicații

- **exemplu:** alocarea dinamică a unui tablou bi-dimensional

tablou_bidimensional.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
  int L, C, i, j;
  int **p; // Adresa matrice

  printf("Nr de linii L = "); scanf("%d", &L);
  printf("Nr de coloane C = "); scanf("%d", &C);

  p = (int**) malloc(L * sizeof(int*));
  printf("Sizeof(int*) = %d \n",sizeof(int*));
  printf("Pointerul p contine adresa %d \n",p);

  for (i = 0; i < L; i++)
  {
    p[i] = calloc(C, sizeof(int));
    printf("Linia %d incepe la %d \n",i,p[i]);
  }

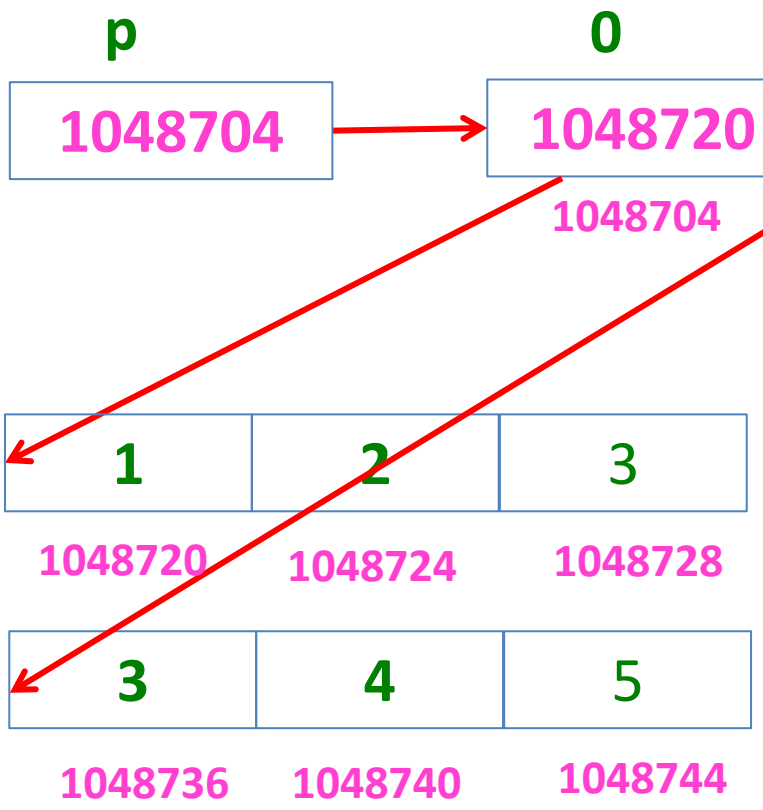
  for (i = 0; i < L; i++) {
    for (j = 0; j < C; j++) {
      p[i][j] = L * i + j + 1;
      printf("Adresa lui p[%d][%d] este = %d \n",i,j,&p[i][j]);
    }
  }

  for (i = 0; i < L; i++) {
    for (j = 0;j < C; j++) {
      printf("%d ", p[i][j]);
    }
    printf("\n");
```

```
Nr de linii L = 2
Nr de coloane C = 3
Sizeof(int*) = 8
Pointerul p contine adresa 1048704
Linia 0 incepe la 1048720
Linia 1 incepe la 1048736
Adresa lui p[0][0] este = 1048720
Adresa lui p[0][1] este = 1048724
Adresa lui p[0][2] este = 1048728
Adresa lui p[1][0] este = 1048736
Adresa lui p[1][1] este = 1048740
Adresa lui p[1][2] este = 1048744
1 2 3
3 4 5
```

Alocare dinamică – aplicații

- **exemplu:** alocarea dinamică a unui tablou bi-dimensional




```
Nr de linii L = 2
Nr de coloane C = 3
Sizeof(int*) = 8
Pointerul p contine adresa 1048704
Linia 0 incepe la 1048720
Linia 1 incepe la 1048736
Adresa lui p[0][0] este = 1048720
Adresa lui p[0][1] este = 1048724
Adresa lui p[0][2] este = 1048728
Adresa lui p[1][0] este = 1048736
Adresa lui p[1][1] este = 1048740
Adresa lui p[1][2] este = 1048744
1 2 3
3 4 5
```

Alocare dinamică – aplicații

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p,n,i,j,c;
    scanf("%d",&n);
    p=malloc(n*n*sizeof(int));
    if (p==NULL)
        printf("\nEroare de alocare.");
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
        {
            printf("[%d][%d]=",i,j);
            scanf("%d",p+i*n+j);
        }
    scanf("%d",&c);
    for(i=0;i<n;i++)
        {for(j=0;j<n;j++)
            printf("[%d][%d]=%d ",i,j,c**(p+i*n+j));
            printf("\n");
        }
    free(p);
    return 0;
}
```

Enunț?

Alocare dinamică – aplicații

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p,n,i,j,c;
    scanf("%d",&n);
    p=malloc(n*n*sizeof(int));
    if (p==NULL)
        printf("\nEroare de alocare.");
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
        {
            printf("[%d][%d]=",i,j);
            scanf("%d",p+i*n+j);
        }
    scanf("%d",&c);
    for(i=0;i<n;i++)
        {for(j=0;j<n;j++)
            printf("[%d][%d]=%d ",i,j,c**(p+i*n+j));
            printf("\n");
        }
    free(p);
    return 0;
}
```

Enunț:

Matrice pătratică/ înmulțirea cu un scalar:
1. citirea dimensiunii n,
2. citirea celor n*n elemente
3. citirea unui întreg c
4. scrierea elementelor matricei înmulțite cu valoarea lui c

Alocare dinamică – aplicații

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p,*q,n,i,j,c;
    scanf("%d",&n);
    p=malloc(n*n*sizeof(int));
    q=malloc(n*n*sizeof(int));
    if (p==NULL || q==NULL)
        printf("\nEroare de alocare.");
    else
    {
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
        { scanf("%d",p+i*n+j);}
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
        { scanf("%d",q+i*n+j);}
    for(i=0;i<n;i++)
        {for(j=0;j<n;j++)
          printf("%d ",*(p+i*n+j)+*(q+i*n+j));
          printf("\n");
        }
    }
    free(p); free(q);
    return 0;
}
```

Enunț?

Alocare dinamică – aplicații

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p,*q,n,i,j,c;
    scanf("%d",&n);
    p=malloc(n*n*sizeof(int));
    q=malloc(n*n*sizeof(int));
    if (p==NULL || q==NULL)
        printf("\nEroare de alocare.");
    else
    {
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
        { scanf("%d",p+i*n+j);}
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
        { scanf("%d",q+i*n+j);}
    for(i=0;i<n;i++)
        {for(j=0;j<n;j++)
          printf("%d ",*(p+i*n+j)+*(q+i*n+j));
          printf("\n");
        }
    }
    free(p); free(q);
    return 0;
}
```

Enunț:

Suma a două matrice pătratice

Alocare statică, alocare dinamică

	sir	matrice	sir	matrice	sir	matrice
variabile	A[i],i, n	A[i][j],i,j, n,m	A[i],i,n, *p=A	A[i][j],i,j,n,m, *p=A	i,n, *p; Functii alocare dinamica	i,j,n,m,*p Functii alocare dinamica
adresa	&A[i]	&A[i][j]	p+i	*(p+i)+j	p+i	*(p+i)+j
valoare	A[i]	A[i][j]	*(p+i)	*(*(p+i)+j)	*(p+i)	*(*(p+i)+j)

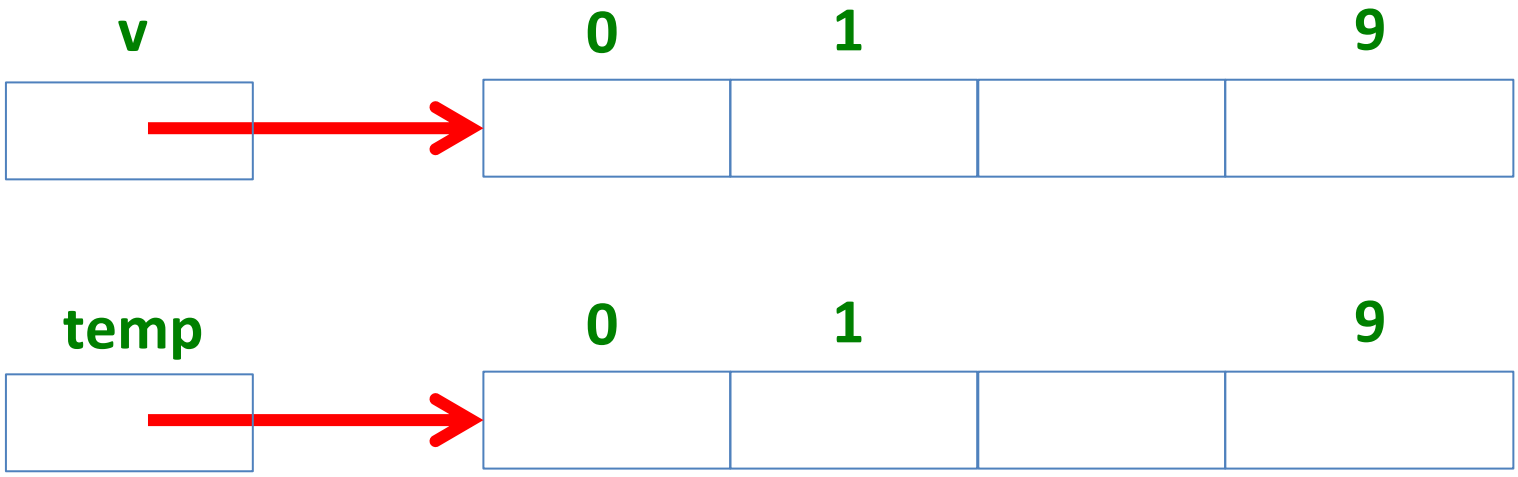
Alocare dinamică – avantaje + dezavantaje

- **avantaje:**
  - ✓ durata de viață: putem controla când are loc alocarea și dezalocarea memoriei
  - ✓ memoria: dimensiunea memoriei alocată poate fi controlată în timpul execuției programului. Spre exemplu un tablou poate fi alocat astfel încât are să aibă dimensiunea identică cu cea a unui tablou specificat în timpul execuției programului

- **dezavantaje:**
  - ✓ mai mult de codat: alocarea memoriei trebuie făcută explicit în cod
  - ✓ posibile bug-uri: lucrul cu pointerii (crash-uri de memorie)

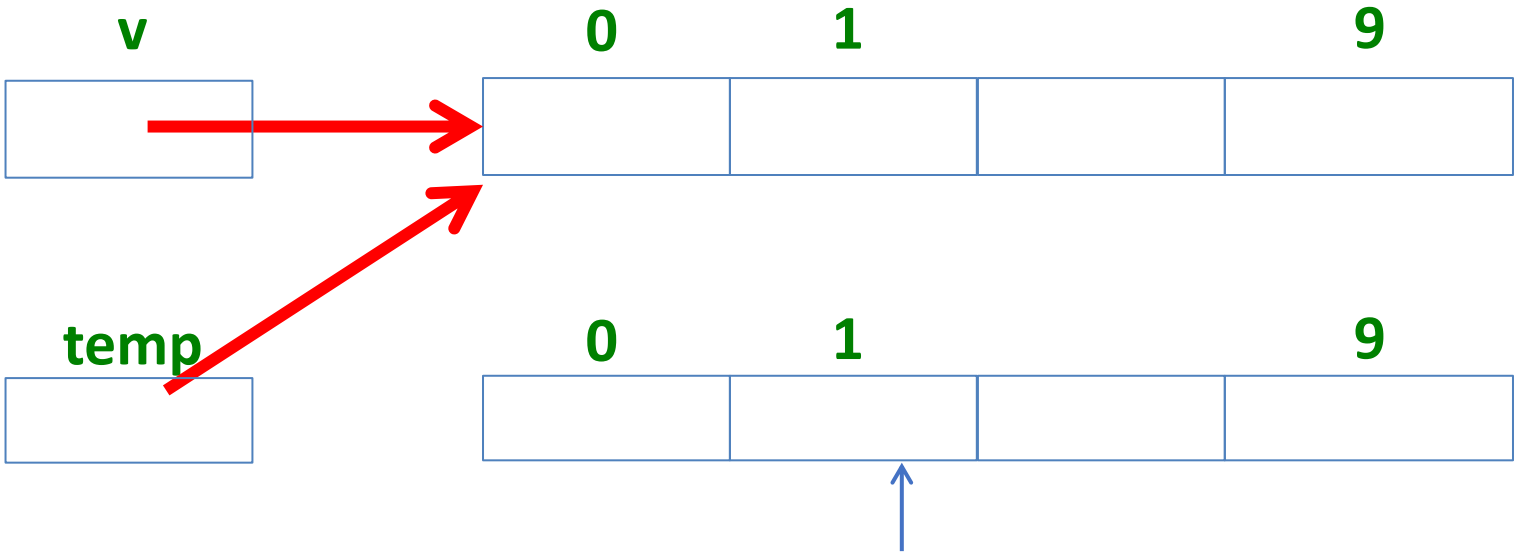
Alocare dinamică – *greșeli*

```
int *v, *temp;
v = (int*) malloc(10*sizeof(int));
temp = (int*) malloc(10*sizeof(int));
temp = v; //fac o copie a lui v in temp
```



Alocare dinamică – *greșeli*

```
int *v, *temp;
v = (int*) malloc(10*sizeof(int));
temp = (int*) malloc(10*sizeof(int));
temp = v; //fac o copie a lui v in temp
```



Zonă marcată de sistemul de operare ca fiind ocupată dar inutilizabilă întrucât am “pierdut” adresa de început a blocului.
(zonă orfană de memorie)

Alocare dinamică – *greșeli*

```
void f(…){
int *p = (int*) malloc(10*sizeof(int));
…
}
```



p este variabilă locală funcției f și va fi distrusă la ieșirea din funcție. Totuși memoria rămâne alocată și inutilizabilă (zonă orfană de memorie)

```
void f(…){
int *p = (int*) malloc(10*sizeof(int));
free(p); //eliberare memorie
}
```

Alocare dinamică – *greșeli*

```
int v[200];

free(v);
```

/*v e alocat static, pot elibera cu funcția **free** numai blocuri de memorie alocate dinamic*/

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int v[100];

    free(v);

    return 0;
}
```

```
main.c: In function 'main':
main.c:8:9: warning: 'free' called on unallocated object 'v' [-Wfree-nonheap-object]
    8 |         free(v);
      |         ^~~~~~~
main.c:6:13: note: declared here
    6 |         int v[100];
      |             ^
munmap_chunk(): invalid pointer


...Program finished with exit code 134
```

Cursul 7

1. Legătura dintre pointeri și tablouri
2. Aritmetica pointerilor
3. Alocare dinamică a memoriei

Cursul 8

1. Clase de memorie
2. Șiruri de caractere. Funcții specifice de manipulare